Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

М.Түвшинжаргал

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Бакалаврын төгсөлтийн ажил

Даланзадгад хот

Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

Мэдээллийн технологийн тэнхим

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

Мэргэжлийн индекс: D061304
Мэргэжил: Мэдээллийн технологи
Удирдагч: Доктор Д.Эрдэнэтуяа
Зөвлөгч:
Гүйцэтгэгч: М.Түвшинжаргал

Даланзадгад хот
2026 он

Батлав. Мэдээллийн технологийн тэнхимын эрхлэгч:

..... //

Удирдагч: ..... /Доктор Д.Эрдэнэтуяа/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Төгсөлтийн ажлын сэдэв:
Монгол: Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт
Англи: Design and Implementation of a Three-Tier Sports Hall Booking System Based on Cloud Environment

Төслийн зорилго: Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын цаг захиалгыг цахимжуулах, хэрэглэгчдэд хялбар, хурдан захиалга хийх боломж олгох Flutter мобайл аппликейшн хөгжүүлэх.

Оюутны овог нэр, код: М.Түвшинжаргал/B221940619/

№	Ажлын бүлэг, хэсгийн нэр	эзлэх хувь	дуусах хугацаа
Бүлэг №1. Оршил ба судалгааны үндэслэл			
1	1.1 Судалгааны үндэслэл, асуудлын тодорхойлолт 1.2 Зорилго, зорилтууд 1.3 Ажлын ач холбогдол, хамрах хүрээ	20%	III.13
Бүлэг №2. Онолын үндэс ба технологийн судалгаа			
2	2.1 Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага 2.2 Flutter фреймворк ба Dart хэл 2.3 Firebase бэкэнд үйлчилгээ 2.4 Цаг захиалгын системийн онолын үндэс	30%	IV.10
Бүлэг №3. Системийн зохион байгуулалт ба хэрэгжүүлэлт			
3	3.1 Системийн шаардлагын дүн шинжилгээ 3.2 Системийн архитектур 3.3 Өгөгдлийн сангийн зохион байгуулалт 3.4 Flutter апп-ын хэрэгжүүлэлт 3.5 Туршилт ба үр дүнгийн үнэлгээ	50%	V.22
Ерөнхий дүгнэлт			

Төлөвлөгөөг боловсруулсан оюутан: ..... /М.Түвшинжаргал/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Удирдагчийн гарын үсэг
1	Бүлэг 1: Оршил, судалгааны үндэслэл боловсруулсан	2026-03-13	
2	Бүлэг 2: Онолын үндэс, технологийн судалгаа хийсэн	2026-04-10	
3	Бүлэг 3: Систем зохион байгуулж, Flutter апп хөгжүүлсэн	2026-05-15	
4	Туршилт хийж, дүгнэлт боловсруулсан	2026-05-22	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: ..... /Доктор Д.Эрдэнэтуяа/

ЗӨВШӨӨРӨЛ

Оюутан М.Түвшинжаргалгын бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: ..... //

Зохиогчийн эрх хамгаалал

Миний бие М.Түвшинжаргал, Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

Өмнөговь Технологийн Дээд Сургууль
Мэдээллийн Технологийн Тэнхим

Хураангуй

Үүлэн орчинд суурилсан спорт заалны цаг захиалгын 3 давхаргат системийн загварчлал, хэрэгжилт

М.Түвшинжаргал

Түлхүүр үгс: *Flutter, Firebase, Node.js, Docker, AWS EC2, GitHub Actions, AI чатбот, цаг захиалга, спорт заал, үүлэн орчин, Даланзадгад*

Энэхүү бакалаврын төгсөлтийн ажлын зорилго нь Өмнөговь аймгийн Даланзадгад хотын таван спорт заалд зориулсан үүлэн орчинд суурилсан цаг захиалгын мобайл аппликейшн хөгжүүлэх явдал юм.

Систем нь **Flutter** (Android/iOS), **Node.js/Express** бэкэнд, **Firebase** (Auth, Firestore, Cloud Functions), **Docker/AWS EC2** дэд бүтэц болон **AI чатбот** (Claude Haiku, Groq Llama) гэсэн таван бүрэлдэхүүнтэй гурван давхаргат архитектурт суурилна. Бэкэнд нь **AWS EC2** (ap-southeast-1) дээр Docker контейнераар ажиллаж, **GitHub Actions** CI/CD хоолойгоор автоматаар байршуулагддаг.

Үндсэн функцүүд: email/нууц үгийн нэвтрэлт (Nodemailer 6 оронтой кодын баталгаажуулалттай), цаг захиалах болон цуцлах (DBK-xxxxx кодтой), байгууллагуудын тогтмол захиалга (`fixed_bookings`), 1 цагийн өмнөх локал мэдэгдэл, AI чатбот.

Функциональ туршилтаар 20 тест тохиолдол бүгд амжилттай давж, хэрэглэхэд хялбар байдлын нийт оноо **4.3/5.0** болсон.

Keywords: *Flutter, Firebase, Node.js, Docker, AWS EC2, GitHub Actions, AI chatbot, booking system, sports hall, cloud, Dalanzadgad*

This bachelor's thesis develops a cloud-based three-tier sports hall booking mobile application for Dalanzadgad city, Ömnögovi Province. The system comprises a Flutter mobile client (Android/iOS), a Node.js/Express backend on AWS EC2 via Docker, Firebase services (Authentication, Firestore, Cloud Functions), and dual AI chatbot (Anthropic Claude Haiku 4.5 / Groq Llama 3.1-8b). Automated deployment is handled by GitHub Actions CI/CD. All 20 test cases passed with a usability score of 4.3/5.0.

Талархал

Энэхүү бакалаврын төгсөлтийн ажлыг гүйцэтгэхэд чиглэл заагч, зөвлөгөө өгч, тасралтгүй дэмжиж ирсэн удирдагч багш **Доктор Д.Эрдэнэтуяа** багш танд гүн талархал илэрхийлье.

Дөрвөн жилийн турш мэдлэг, чадвар, судалгааны арга барилыг эзэмшихэд тусалсан **Өмнөговь Технологийн Дээд Сургуулийн** бүх багш нарт талархал илэрхийлж байна.

Мөн судалгааны явцад туршилтад оролцож, санал хүсэлтээ өгсөн **Өмнөговь аймгийн Даланзадгад хотын спорт заалнуудын** захиралууд болон ажилчдад талархал илэрхийлье.

Эцэст нь, тасралтгүй урам өгч, дэмжиж ирсэн гэр бүлийнхэндээ болон хамт сурдаг нөхдөдөө чин сэтгэлийн талархлаа илэрхийлье.

Товчилсон үгс

UI	User Interface (Хэрэглэгчийн интерфэйс)
UX	User Experience (Хэрэглэгчийн туршлага)
API	Application Programming Interface (Аппликейшн програмчлалын интерфэйс)
SDK	Software Development Kit (Програм хангамжийн хөгжүүлэлтийн хэрэгсэл)
IDE	Integrated Development Environment (Нэгдсэн хөгжүүлэлтийн орчин)
OS	Operating System (Үйлдлийн систем)
iOS	iPhone Operating System (Apple-ийн мобайл үйлдлийн систем)
APK	Android Package Kit (Android апп-ын суулгац файл)
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation (Өгөгдөл солилцооны формат)
JWT	JSON Web Token (Нэвтрэлтийн токен формат)
REST	Representational State Transfer
MVC	Model-View-Controller (Загвар-Харагдац-Удирдагч)
DB	Database (Өгөгдлийн сан)
SQL	Structured Query Language
NoSQL	Not only SQL (Уламжлалт бус өгөгдлийн сан)
OTP	One-Time Password (Нэг удаагийн нууц үг)
FCM	Firebase Cloud Messaging (Firebase-ийн мэдэгдлийн үйлчилгээ)
CI/CD	Continuous Integration / Continuous Deployment (Тасралтгүй нэгтгэлт / байршуулалт)
AWS	Amazon Web Services (Амазоны үүлэн үйлчилгээ)
EC2	Elastic Compute Cloud (AWS-ийн виртуал сервер үйлчилгээ)
ECR	Elastic Container Registry (AWS-ийн Docker дүрс хадгалах сан)
VPC	Virtual Private Cloud (Виртуал хувийн сүлжээ)
IAM	Identity and Access Management (AWS нэвтрэлт, эрх удирдлага)
AI	Artificial Intelligence (Хиймэл оюун ухаан)
LLM	Large Language Model (Том хэлний загвар)
SaaS	Software as a Service (Үйлчилгээ хэлбэрийн програм хангамж)
BaaS	Backend as a Service (Үйлчилгээ хэлбэрийн бэкэнд)
UML	Unified Modeling Language (Нэгдсэн загварчлалын хэл)
QR	Quick Response (Шуурхай хариу кодчилол)

Гарчиг

Зохиогчийн эрх хамгаалал i

Хураангуй ii

Талархал iii

Товчилсон үгс iv

Удиртгал x

1 ОНОЛЫН ҮНДЭС БА ТЕХНОЛОГИЙН СУДАЛГАА 1
1.1 Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага . . . 1
1.2 Flutter фреймворк ба Dart хэл . . . 1
1.2.1 Flutter-ийг сонгосон үндэслэл . . . 1
1.2.2 Flutter-ийн тухай . . . 2
1.2.3 Dart програмчлалын хэл . . . 2
1.2.4 StatefulWidget ба setState-д суурилсан төлвийн удирдлага . . 2
1.3 Firebase бэкэнд үйлчилгээ . . . 3
1.3.1 Cloud Firestore . . . 3
1.3.2 Firebase Authentication . . . 4
1.3.3 Firebase Cloud Functions . . . 4
1.4 Node.js болон Express.js бэкэнд . . . 4
1.4.1 Node.js . . . 4
1.4.2 Express.js фреймворк . . . 4
1.4.3 REST API архитектур . . . 5
1.5 Docker контейнержуулалт . . . 5
1.6 AWS дэд бүтэц . . . 6
1.6.1 Amazon EC2 . . . 6
1.6.2 Amazon ECR . . . 6
1.7 GitHub Actions CI/CD . . . 6
1.8 AI үйлчилгээнүүд . . . 6
1.8.1 Anthropic Claude Haiku . . . 6
1.8.2 Groq Llama 3.1-8b . . . 7
1.9 flutter_local_notifications . . . 7
1.10 Технологийн стекийн нэгтгэсэн дүгнэлт . . . 7
1.11 Ижил төстэй системүүдийн дүн шинжилгээ . . . 7
1.11.1 Монгол улсын цаг захиалгын системүүд . . . 7
1.11.2 Судалгааны дүгнэлт . . . 8

2 СИСТЕМИЙН ЗОХИОН БАЙГУУЛАЛТ БА ХЭРЭГЖҮҮЛЭЛТ 9
2.1 Системийн шаардлагын дүн шинжилгээ . . . 9
2.1.1 Функциональ шаардлага . . . 9
2.1.2 Функциональ бус шаардлага . . . 9
2.2 Системийн UML диаграммууд . . . 9

2.2.1 Use Case Diagram (Хэрэглэгчийн тохиолдлын диаграмм) . . . 9
2.2.2 Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram) 10
2.2.3 Захиалга үүсгэх дараалын диаграмм (Sequence Diagram) . . 10
2.2.4 Email баталгаажуулалтын дараалын диаграмм 10
2.2.5 Firestore өгөгдлийн сангийн схем (ER Диаграмм) 10
2.3 Системийн архитектур 10
2.3.1 Ерөнхий архитектур 10
2.3.2 Үүлэн байршуулалтын (Cloud Deployment) архитектур 11
2.3.3 CI/CD Хоолойн архитектур (GitHub Actions) 11
2.3.4 Flutter апп-ын бүтэн архитектур 11
2.3.5 API URL-ийн нэг эх сурвалж (lib/config.dart) 12
2.4 Өгөгдлийн сангийн зохион байгуулалт 13
2.4.1 `users` collection 13
2.4.2 `bookings` collection 14
2.4.3 `fixed_bookings` collection 15
2.4.4 `email_verifications` collection 16
2.4.5 Hardcoded спорт заалнуудын жагсаалт 16
2.5 Flutter апп-ын хэрэгжүүлэлт 17
2.5.1 Дулаан говийн дизайн (Warm Desert Theme) 17
2.5.2 4 tab-тай доод навигаци 18
2.5.3 Email/нууц үгийн нэвтрэлт ба баталгаажуулалт 20
2.5.4 Нүүр дэлгэц — Спорт заалнуудын жагсаалт 22
2.5.5 Цагийн хуваарь ба захиалгын дэлгэц 22
2.5.6 Локал мэдэгдлийн хэрэгжүүлэлт 24
2.6 Node.js/Express бэкэндийн хэрэгжүүлэлт 26
2.6.1 Бэкэндийн бүтэц 26
2.6.2 Үндсэн API endpoint-ууд 26
2.6.3 Cron ажил — Дуусгавар болсон захиалгыг автоматаар шинэчлэх 26
2.7 AI чатботын хэрэгжүүлэлт 27
2.7.1 Архитектур — Үндсэн болон нөөц горим 27
2.7.2 Firebase Cloud Functions — Claude Haiku 27
2.7.3 Express бэкэнд — Groq Llama нөөц горим 28
2.8 Docker контейнержуулалт ба AWS байршуулалт 28
2.8.1 Docker Image 28
2.8.2 GitHub Actions CI/CD Workflow 29
2.9 Туршилт ба үр дүнгийн үнэлгээ 30
2.9.1 Функциональ туршилт 30
2.9.2 Гүйцэтгэлийн хэмжилт 31
2.9.3 CI/CD хоолойн туршилт 31

3 СИСТЕМИЙН ТУРШИЛТ БА ҮНЭЛГЭЭ 33
3.1 Туршилтын орчин, аргачлал 33
3.2 Функциональ туршилт 33
3.2.1 Туршилтын тохиолдлууд 33
3.2.2 Байгууллагын тогтмол захиалгын туршилт 34
3.3 Нэгтгэлийн туршилт 34
3.4 Гүйцэтгэлийн туршилт 35
3.4.1 Хариу цагийн хэмжилт 35
3.4.2 Cron ажлын туршилт 35
3.5 Хэрэглэхэд хялбар байдлын үнэлгээ 35
3.5.1 Хэрэглэгчийн туршилт 35

3.5.2 UI/UX үнэлгээ 36
3.6 Аюулгүй байдлын шалгалт 36
3.7 Туршилтын нэгтгэсэн дүн 37

Дүгнэлт **38**

A Аппликейшний дэлгэцийн зураглал **40**

B Үндсэн тохиргооны файлууд **43**

Ном зүй **46**

Зургийн жагсаалт

2.1 Use Case Diagram — Хэрэглэгч болон Админы харилцаа 11
2.2 Цаг захиалах үйл ажиллагааны диаграмм 12
2.3 Захиалга үүсгэх дараалын диаграмм 13
2.4 Email баталгаажуулалтын дараалын диаграмм (Firebase Auth + Nodemailer) 14
2.5 Firestore өгөгдлийн сангийн схем 15
2.6 Системийн гурван давхаргат үүлэн архитектурын ерөнхий диаграмм 16
2.7 Үүлэн байршуулалтын архитектур — AWS, Firebase, CI/CD 17
2.8 GitHub Actions CI/CD хоолойн дэлгэрэнгүй диаграмм 18
2.9 Flutter аппликейшний бүтэн давхаргат архитектур 19
2.10 Email/нууц үгийн нэвтрэлт ба баталгаажуулалтын дэлгэцүүд 21
2.11 Нүүр дэлгэц ба захиалгын жагсаалт 22
2.12 Цагийн хуваарь ба захиалгын дэлгэцүүд 24
2.13 Мэдэгдлүүдийн дэлгэц — захиалгын сануулга 25
2.14 AI Туслах дэлгэц — спорт заалтай холбоотой асуулт асуух 29

A.1 Нэвтрэх дэлгэц (зүүн) ба бүртгэлийн дэлгэц (баруун) 40
A.2 Нүүр дэлгэц — таван спорт заалны жагсаалт 41
A.3 Цагийн хуваарийн дэлгэц (зүүн) ба захиалга баталгаажуулах дэлгэц (баруун) 42
A.4 Захиалгын жагсаалтын дэлгэц (зүүн) ба AI чатботын дэлгэц (баруун) 42

Хүснэгтийн жагсаалт

1 Одоогийн цаг захиалгын асуудлууд ба шийдэл x

1.1 Мобайл аппликейшн хөгжүүлэлтийн арга хандлагуудын харьцуулалт 1
1.2 Flutter болон React Native харьцуулалт 1
1.3 Node.js/Express REST API endpoint-уудын жагсаалт 5
1.4 Ашигласан технологийн стекийн нэгтгэсэн хүснэгт 8
1.5 Цаг захиалгын системүүдийн харьцуулсан шинжилгээ 8

2.1 Функциональ бус шаардлагуудын хүснэгт 10
2.2 Функциональ туршилтын үр дүн 31

3.1 Функциональ туршилтын тохиолдлууд 33
3.2 Нэгтгэлийн туршилтын үр дүн 34
3.3 Системийн хариу цагийн хэмжилт (секунд) 35
3.4 Хэрэглэхэд хялбар байдлын үнэлгээ 36
3.5 UI/UX үнэлгээ (1–5 шкал) 36
3.6 Аюулгүй байдлын шалгалтын үр дүн 37
3.7 Туршилтын нийт үр дүн 37
3.8 Зорилтуудын биелэлт 39

УДИРТГАЛ

Сэдэв сонгосон үндэслэл

Мэдээллийн технологийн хурдацтай хөгжил нь орчин үеийн иргэдийн өдөр тутмын амьдралыг ихээхэн өөрчилж байна. Цахим захиалга, онлайн үйлчилгээ нь дэлхийн өнцөг булан бүрт нэвтэрч, хүмүүсийн цаг хугацааг хэмнэж, үйлчилгээний чанарыг дээшлүүлж байна. Statista судалгааны байгууллагын 2024 оны тайланд дурдснаар дэлхийн мобайл аппликейшний зах зээлийн хэмжээ 2028 он гэхэд 756 тэрбум ам.долларт хүрнэ гэж таамаглаж байна [1]. Гэвч Монгол улсын орон нутгийн хотуудад, тухайлбал Өмнөговь аймгийн Даланзадгад хотод, уламжлалт үйлчилгээний хэлбэр давамгайлсаар байна.

Даланзадгад хотод өнөөдрийн байдлаар таван спорт заал тамирын цогцолбор үйл ажиллагаа явуулж байгаа бөгөөд иргэд цаг захиалахдаа дараах бэрхшээлтэй тулгардаг:

Хүснэгт 1: Одоогийн цаг захиалгын асуудлууд ба шийдэл

#	**Тулгамдаж буй асуудал**	**Системийн шийдэл**
1	Утсаар захиалахад шугам завгүй байх, буруу ойлголцол үүсэх	Онлайн захиалга — цагийн хуваарь бодит цагаар харагдах
2	Биечлэн очиж захиалахад цаг, зардал зарцуулагдах	Хаанаас ч гэсэн мобайл аппаар захиалах боломж
3	Боломжтой цагийн мэдээлэл тодорхойгүй, ил тод бус байх	Firestore бодит цагийн шинэчлэлтээр цагийн хуваарь нэн даруй харагдах
4	Захиалгаа цуцлах, өөрчлөх боломж хязгаарлагдмал байх	Апп дотроос нэг дарахад захиалга цуцлах функц
5	Захиалгын баталгаажуулалт, сануулга авах боломжгүй байх	DBK-xxxxx захиалгын код + 1 цагийн өмнөх локал мэдэгдэл

Эдгээр асуудлыг шийдвэрлэхийн тулд орчин үеийн мобайл болон **үүлэн технологи**-г ашиглан цаг захиалгын систем хөгжүүлэх нь чухал шаардлага болж байна. **Flutter** фреймворк нь нэг удаа бичиж Android болон iOS платформд нэгэн зэрэг гаргах боломж олгодог тул хөгжүүлэлтийн зардал, хугацааг мэдэгдэхүйц бууруулдаг. **Firebase** нь Google-ийн **үүлэн** бэкэнд үйлчилгээ бөгөөд бодит цагийн өгөгдөл солилцоо, найдвартай нэвтрэлт, тасралтгүй ажиллагааг хангадаг. Харин **Node.js/Express** бэкэнд нь **Docker** контейнержуулалтаар **AWS EC2 үүлэн серверт** байршиж, **GitHub Actions** CI/CD хоолойгоор автоматаар нийтлэгддэг тул систем нь бодитоор **үүлэн орчинд** (cloud-native) ажилладаг.

Зорилго

Зорилго: Өмнөговь аймгийн Даланзадгад хотын таван спорт заалны цаг захиалгыг цахимжуулах, хэрэглэгчдэд хялбар, хурдан, ил тод байдлаар захиалга хийх боломж олгох **Flutter** мобайл аппликейшн болон **Node.js/Express** бэкэнд бүхий гурван давхаргат системийг **үүлэн орчинд** (AWS EC2) хөгжүүлж, байршуулах.

Зорилт

Уг зорилгод хүрэхийн тулд дараах зорилтуудыг дэвшүүлж байна:

1. Даланзадгад хотын спорт заалнуудын цаг захиалгын одоогийн нөхцөл байдал, иргэдийн хэрэгцээг судлах, тулгамдаж буй асуудлуудыг тогтоох;
2. Цаг захиалгын мобайл аппликейшний функциональ болон функциональ бус шаардлагын дүн шинжилгээ хийж, Use Case Diagram, Activity Diagram, Sequence Diagram боловсруулах;
3. Flutter болон Firebase технологийг ашиглан гурван давхаргат (клиент–бэкэнд–үүлэн дэд бүтэц) системийн архитектур зохион байгуулах;
4. Material Design 3 стандартад нийцсэн, дулаан говийн дизайн бүхий мобайл аппликейшн хэрэгжүүлэх;
5. Node.js/Express REST API бэкэнд хөгжүүлж, Docker контейнержуулалтаар AWS EC2-д байршуулах;
6. GitHub Actions CI/CD хоолойгоор бэкэндийн автоматжуулсан байршуулалтыг хэрэгжүүлэх;
7. Функциональ, нэгтгэлийн болон гүйцэтгэлийн туршилт хийж, хэрэглэхэд хялбар байдлыг үнэлэх.

Судлагдсан байдал

Спорт заал болон үйлчилгээний цаг захиалгын мобайл аппликейшн хөгжүүлэлтийн чиглэлд олон улсын болон дотоодын судлаачид тодорхой хэмжээний ажил хийсэн байна.

Монгол улсын хэмжээнд **eZaal** (DAZO LLC) нь спортын байгууламжийн захиалгыг нэгтгэсэн томоохон платформ бөгөөд Google Play дээр 10,000+ татагдсан [2]. Гэхдээ зөвхөн Android платформд зориулагдсан, AI чатбот байхгүй, орон нутгийн хотуудад үйлчилгээ хүрдэггүй. Kassim et al. (2022) судалгаанд Flutter-г ашиглан sport facility booking систем хөгжүүлсэн туршлагыг дүрсэлсэн бөгөөд Firebase-тэй хослуулсан нь хөгжүүлэлтийн хугацааг 40%-иар бууруулсан гэж тэмдэглэжээ [3].

Монголын эрдэм шинжилгээний хэмжээнд Амарсайханы Билгүүн [4] мобайл аппликейшн хөгжүүлэлтийн аргачлалын асуудлыг судалсан бол Т.Амартүвшин, Б.Зоригтбаатар нар [5] мобайл апп хөгжүүлэлтийн онолын үндсийг тодорхойлсон байна. Гэхдээ тухайлан **үүлэн орчинд суурилсан**, **гурван давхаргат архитектур** бүхий спорт заалны цаг захиалгын тусгай системийн талаарх судалгаа орон нутгийн хэмжээнд хийгдээгүй байна.

Иймд энэхүү ажил нь орон нутгийн хэрэгцээнд нийцсэн, cross-platform, AI чатбот болон CI/CD автоматжуулалт бүхий орчин үеийн технологийн бүрэн стек ашигласан шинэлэг шийдлийг санал болгож байна.

Шинэлэг тал

Энэхүү ажлын шинэлэг тал нь дараахид оршино:

1. **Гурван давхаргат үүлэн архитектур:** Flutter клиент — Node.js/Express бэкэнд (AWS EC2) — Firebase/Google Cloud дэд бүтэц гэсэн тодорхой давхаргаар системийг зохион байгуулсан бөгөөд систем нь бодитоор үүлэн орчинд ажилладаг;

2. **Docker + AWS CI/CD автоматжуулалт:** GitHub Actions ашиглан код шинэчлэгдэх бүрт Docker дүрс build хийж, AWS ECR-д push хийж, EC2-д автоматаар байршуулдаг хоолой хэрэгжүүлсэн (3–4 минут);

3. **Давхар горимын AI чатбот:** Anthropic Claude Haiku 4.5 (Firebase Cloud Functions, үндсэн горим) болон Groq Llama 3.1-8b (Node.js бэкэнд, нөөц горим) загваруудыг нөөцлөлтийн горимоор нэгтгэж, найдвартай AI чатбот хэрэгжүүлсэн;

4. **Байгууллагын тогтмол захиалга:** Даланзадгад хотын засгийн газар, компаниудын долоо хоногийн тогтмол цагийг `fixed_bookings` цуглуулгаар удирдах тусгай функц хэрэгжүүлсэн;

5. **Орон нутгийн хэрэгцээ:** Даланзадгад хотын таван спорт заалд тохирсон, Монгол хэлтэй, дулаан говийн дизайн бүхий мобайл аппликейшнийг Монгол улсын орон нутгийн хотод анх удаа хэрэгжүүлсэн.

Судалгааны хамрах хүрээ

Хамрах хүрээ: Энэхүү ажлын хүрээнд дараахи зүйлс орно:

- Иргэдэд зориулсан мобайл апп (Android 8.0+ болон iOS 13.0+);
- Firebase дээр суурилсан email/нууц үгийн нэвтрэлт (Nodemailer 6 оронтой кодын баталгаажуулалттай, rollback механизмтай), 4 Firestore collection;
- Node.js/Express REST API бэкэнд (5 endpoint);
- Docker контейнержуулалт, AWS EC2/ECR байршуулалт (ap-southeast-1);
- GitHub Actions CI/CD автоматжуулалт;
- AI чатботын үйлчилгээ (Claude Haiku 4.5 + Groq Llama 3.1-8b);
- Локал мэдэгдлийн систем (`flutter_local_notifications`).

Хамрагдахгүй зүйлс: Онлайн төлбөрийн систем (QPay, SocialPay), GPS байршлын нарийн функц, администраторын дэлгэц.

Ач холбогдол

Практик ач холбогдол: Даланзадгад хотын иргэдэд спорт заал захиалах үйл явцыг хялбарчилж, цаг хугацааг хэмнэнэ. Спорт заалны ажилтнуудад захиалгын менежментийг автоматжуулж, ажлын ачааллыг бууруулна. Цахим захиалга нэвтэрснээр буруу ойлголцол, шугам завгүй байхтай холбоотой асуудлуудыг арилгаж, захиалгын ил тод байдлыг хангана.

Ажлын бүтэц

Энэхүү бакалаврын төгсөлтийн ажил дараахи бүтэцтэй:

Удиртгал: Судалгааны үндэслэл, зорилго, зорилт, судлагдсан байдал, шинэлэг тал, хамрах хүрээ, ач холбогдлыг тодорхойлсон.

1-р бүлэг — Онолын үндэс ба технологийн судалгаа: Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага, Flutter/Dart фреймворк, Firebase үйлчилгээ, Node.js/Express, Docker, AWS (EC2/ECR), GitHub Actions CI/CD, AI (Claude

Haiku, Groq Llama) технологиудын онолын үндсийг судалж, ижил төстэй дотоод болон гадаадын системүүдтэй харьцуулсан.

2-р бүлэг — Системийн зохион байгуулалт ба хэрэгжүүлэлт: Функциональ болон функциональ бус шаардлагын дүн шинжилгээ, Use Case Diagram, Activity Diagram, Sequence Diagram, системийн гурван давхаргат архитектур, үүлэн байршуулалтын архитектур, CI/CD хоолой, Flutter апп болон Node.js бэкэндийн дэлгэрэнгүй хэрэгжүүлэлт, Firestore өгөгдлийн загвар.

3-р бүлэг — Системийн туршилт ба үнэлгээ: Функциональ туршилт (10 тохиолдол), нэгтгэлийн туршилт (4 тохиолдол), гүйцэтгэлийн хэмжилт (6 үзүүлэлт), хэрэглэхэд хялбар байдлын үнэлгээ (8 оролцогч), аюулгүй байдлын шалгалт.

Дүгнэлт: Ажлын нийт үр дүн, хүрсэн зорилтуудын биелэлт болон цаашдын хөгжлийн чиглэлийг дүгнэсэн.

БҮЛЭГ 1

ОНОЛЫН ҮНДЭС БА ТЕХНОЛОГИЙН СУДАЛГАА

1.1 Мобайл апп хөгжүүлэлтийн орчин үеийн арга хандлага

Мобайл аппликейшн хөгжүүлэлтэд өнөөдөр гурван үндсэн арга хандлага байдаг: **Native**, **Hybrid**, **Cross-platform**. Эдгээрийг харьцуулбал:

Хүснэгт 1.1: Мобайл аппликейшн хөгжүүлэлтийн арга хандлагуудын харьцуулалт

Шалгуур	**Native**	**Hybrid**	**Cross-platform**
Хэл/хэрэгсэл	Swift, Kotlin/Java	HTML, JS, CSS	Flutter, React Native
Гүйцэтгэл	Хамгийн өндөр	Доогуур	Native-д ойр
Нэг кодын бааз	Үгүй	Тийм	Тийм
UI нийцтэй байдал	Платформ бүрт ялгаатай	WebView хязгаарлагдмал	Бүрэн нийцтэй
Хөгжүүлэлтийн зардал	Өндөр (×2 баг)	Бага	Дунд
Энэ ажилд	—	—	**Flutter**

Энэхүү ажилд cross-platform хандлагыг сонгосон бөгөөд үүний тулд **Flutter** фреймворкийг ашигласан. Flutter нь Google-ийн боловсруулсан, Dart хэл дээр суурилсан, Android болон iOS платформд нэгэн зэрэг ажиллах чадвартай, гүйцэтгэлийн хувьд native-д ойр шийдэл юм [6]. Cross-platform хандлагыг сонгосноор нэг кодын баазаас хоёр платформ дэмжих боломжтой бөгөөд хөгжүүлэлтийн нийт хугацаа, зардлыг мэдэгдэхүйц бууруулдаг.

1.2 Flutter фреймворк ба Dart хэл

1.2.1 Flutter-ийг сонгосон үндэслэл

Cross-platform хандлагад **Flutter** болон **React Native** хоёр нь гол өрсөлдөгчид юм. Энэхүү ажилд Flutter-ийг сонгосон шалтгааныг дараах харьцуулалтаар тодорхойлж болно:

Хүснэгт 1.2: Flutter болон React Native харьцуулалт

Шалгуур	**Flutter**	**React Native**
Render engine	Skia/Impeller (өөрийн)	Native bridge (платформын)
Хэл	Dart	JavaScript/TypeScript
UI нийцтэй байдал	Платформ бүрт яг ижил	Зарим UI ялгаатай
Гүйцэтгэл	Өндөр (AOT compile)	Бага зэрэг доогуур
Material Design 3	Бүрэн дэмжлэгтэй	Гуравдагч талын багц хэрэгтэй
Widget сан	Баялаг, суурилагдсан	Нэмэлт багц хэрэгтэй
Дүгнэлт	**Энэ ажилд сонгосон**	—

Flutter-ийн **Skia/Impeller** дүрслэлийн хөдөлгүүр нь бүх UI-г платформаас үл хамааран өөрөө зурдаг тул Android болон iOS-д яг ижил харагдах баталгаа олгодог нь энэ ажлын хувьд чухал давуу тал юм.

1.2.2 Flutter-ийн тухай

Flutter нь Google компанийн 2018 онд гаргасан нээлттэй эхийн UI фреймворк юм. Flutter-ийн гол давуу талд дараахи зүйлс орно:

- **Hot Reload** — Кодын өөрчлөлтийг апп-г дахин эхлүүлэхгүйгээр шууд харах боломж;
- **Widget-д суурилсан архитектур** — UI бүрэлдэхүүн хэсэг болгоныг widget хэлбэрээр тодорхойлно;
- **Skia/Impeller render engine** — Бүх UI-г платформаас үл хамааран өөрөө зурдаг тул платформ бүрт ижил харагдах баталгаа олгодог;
- **Нэг кодын бааз** — Android, iOS, Web, Desktop платформд нэгэн зэрэг гаргах боломж.

Flutter нь version 3.x хүрэхдээ Material Design 3 (Material You) стандартыг бүрэн дэмждэг болж, `ThemeData` классаар дамжуулан дүрс, өнгө, хэлбэр бүх зүйлийг системтэй тохируулах боломж нээгдсэн [7]. Энэхүү ажилд Flutter 3.x хувилбарыг ашигласан.

1.2.3 Dart програмчлалын хэл

Dart нь Google-ийн боловсруулсан, object-oriented, strongly typed хэл юм. Flutter хөгжүүлэлтэд Dart ашигладаг бөгөөд дараахи онцлогтой [8]:

- **Async/Await** — Асинхрон үйлдлийг хялбараар бичих, Future болон Stream дэмждэг;
- **Null Safety** — Null pointer алдааг compile time-д илрүүлэх;
- **AOT ба JIT compile** — Хурдан хөгжүүлэлт (JIT) болон хурдан гүйцэтгэл (AOT) нэгэн зэрэг хангах;
- **Stream** — Бодит цагийн өгөгдөл боловсруулах, Firebase Firestore-ийн real-time listener-тэй холбоход тохиромжтой.

Dart хэлний `class` бүтэц нь Flutter-ийн widget системтэй нягт уялддаг. `StatelessWidget` нь өөрчлөлтгүй UI-г илэрхийлж, `StatefulWidget` нь дотоод `State` объектоор дамжуулан өөрчлөгддөг UI-г удирддаг.

1.2.4 StatefulWidget ба setState-д суурилсан төлвийн удирдлага

Flutter апп-д төлвийн удирдлагыг (state management) зөв хийх нь апп-ын найдвартай байдал, гүйцэтгэлд шууд нөлөөлнө. Энэ ажилд **StatefulWidget** болон **setState** хослолыг ашигласан. Энэ нь Flutter-ийн бүтэц дэх хамгийн анхдагч, ойлгомжтой механизм бөгөөд нэмэлт багц шаарддаггүй.

Provider, Riverpod зэрэг өндөр түвшний төлвийн удирдлагын хэрэгслүүдтэй харьцуулахад `setState` нь дараахи тохиолдолд тохиромжтой:

- Апп-ын нарийн төвөгтэй байдал тодорхой хэмжээнд байх;
- Шинэ хөгжүүлэгчдэд ойлгомжтой, уншиж болохуйц байх шаардлагатай;
- Нэмэлт dependency нэмэхгүйгээр апп-ын хэмжээг жижиг байлгах.

Эх код 1.1: StatefulWidget ба setState жишээ

```
class BookingScreen extends StatefulWidget {
  const BookingScreen({super.key});
  @override
  State<BookingScreen> createState() => _BookingScreenState();
}

class _BookingScreenState extends State<BookingScreen> {
  String? selectedTimeSlot;
  bool isLoading = false;

  void _selectSlot(String slot) {
    setState(() {
      selectedTimeSlot = slot;
    });
  }
  // ...
}
```

Апп-ын ерөнхий навигацийг `BottomNavigationBar` widget дамжуулан 4 tab-тай хэрэгжүүлсэн: Нүүр (Home), Захиалгууд (Bookings), Чат (Chat), Профайл (Profile). Идэвхтэй tab-ийн индексийг root StatefulWidget-ийн `setState`-д хадгалдаг.

1.3 Firebase бэкэнд үйлчилгээ

Firebase нь Google-ийн санал болгодог BaaS (Backend as a Service) платформ юм. Уг платформ нь мобайл апп хөгжүүлэгчдэд бэкэнд дэд бүтэц тохируулах шаардлагагүйгээр өгөгдлийн сан, нэвтрэлт зэрэг үйлчилгээг ашиглах боломж олгодог [9].

1.3.1 Cloud Firestore

Cloud Firestore нь NoSQL, баримт жишгийн (document-oriented) өгөгдлийн сан юм [10]. Бодит цагийн шинэчлэлт (real-time listeners), өгөгдлийг offline хадгалах, автомат масштабчлал зэрэг онцлогтой. Firestore нь **Collection** болон **Document** бүтэцтэй:

- **Collection** — Баримтуудын цуглуулга (энэ ажилд: `users`, `bookings`, `fixed_bookings`);
- **Document** — Талбаруудын цуглуулга, JSON форматтай төстэй.

Firestore-ийн security rules нь collection бүрт хэрэглэгч хандах эрхийг нарийн тохируулах боломж олгодог. Энэ ажилд нэвтрэлт хийсэн хэрэглэгч зөвхөн өөрийн захиалгыг унших, шинэ захиалга бичих эрхтэй.

1.3.2 Firebase Authentication

Firebase Authentication нь хэрэглэгчийн нэвтрэлтийг аюулгүй удирдах үйлчилгээ юм [11]. Энэ ажилд **email хаяг болон нууц үгийн** нэвтрэлтийг ашигласан. Хэрэглэгч анх бүртгүүлэх үед Firebase нь `createUserWithEmailAndPassword` функцээр UID үүсгэж, дараа нь Node.js бэкэнд Nodemailer-ийн тусламжтайгаар тухайн email хаягт **6 оронтой баталгаажуулах код** илгээнэ. Хэрэглэгч кодоо амжилттай оруулсны дараа `emailVerified: true` болж нэвтрэх эрх нээгддэг.

Нэвтрэх үед Firebase-ийн `signInWithEmailAndPassword` функцийг ашигладаг бөгөөд Firestore-д хадгалсан `emailVerified` талбарыг шалгаж, баталгаажаагүй хэрэглэгчийн нэвтрэлтийг хориглодог. Энэ нь email хаягийн үнэн зөвийг баталгаажуулахын зэрэгцээ аюулгүй байдлыг нэмэгдүүлдэг.

1.3.3 Firebase Cloud Functions

Firebase Cloud Functions нь серверлэс (serverless) тооцооллын үйлчилгээ юм. Энэ ажилд Cloud Functions-ийг AI чатботын үндсэн endpoint болгон ашигласан: Flutter апп Cloud Function руу хүсэлт илгээхэд уг функц Anthropic-ийн API руу дамжуулан Claude Haiku хариулт авч буцаана. Cloud Functions нь серверийн дэд бүтцийг өөрөө удирддаг тул хөгжүүлэгч апп-ын логикт төвлөрөх боломжтой болдог.

1.4 Node.js болон Express.js бэкэнд

1.4.1 Node.js

Node.js нь Chrome V8 JavaScript engine дээр суурилсан серверийн талын JavaScript ажиллуулах орчин юм [12]. Node.js-ийн гол давуу талууд:

- **Event-driven, non-blocking I/O** — Олон нэгэн зэрэг холболтыг цөөн нөөцөөр боловсруулах чадвартай;
- **npm экосистем** — Дэлхийн хамгийн том нээлттэй эхийн багцын сан;
- **JavaScript нэгдсэн байдал** — Клиент болон серверийн талд нэг хэл ашиглах боломж;
- **Хурдан хөгжүүлэлт** — JSON боловсруулалт, REST API бүтээхэд маш тохиромжтой.

1.4.2 Express.js фреймворк

Express.js нь Node.js дээр суурилсан хөнгөн, уян хатан веб фреймворк юм [13]. RESTful API барихад хамгийн өргөн хэрэглэгддэг Node.js фреймворк бөгөөд дараахи онцлогтой:

- **Middleware систем** — Хүсэлт боловсруулах дамжлагуудыг зохион байгуулах;
- **Router** — URL хаягийн чиглүүлэлтийг модуль болгон хуваах;
- **Хөнгөн бүтэц** — Шаардлагатай зүйлийг л ачаалдаг.

Энэ ажлын бэкэнд нь Express.js ашиглан дараахи REST API endpoint-уудыг хангадаг:

Хүснэгт 1.3: Node.js/Express REST API endpoint-уудын жагсаалт

Арга	Endpoint	Тайлбар
POST	`/api/auth/send-code`	Бүртгэлийн баталгаажуулах код илгээх (Nodemailer)
POST	`/api/auth/verify-code`	6 оронтой кодыг шалгаж, email баталгаажуулах
POST	`/api/users/create`	Шинэ хэрэглэгч Firestore-д бүртгэх
GET	`/api/bookings/:venueId`	Тухайн заалны захиалгуудын жагсаалт авах
POST	`/api/bookings/create`	Шинэ захиалга үүсгэх (давхардал шалгах)
PUT	`/api/bookings/:id/cancel`	Захиалга цуцлах
GET	`/api/schedule/:venueId`	Цагийн хуваарь + fixed_bookings авах
POST	`/api/chat`	Groq Llama 3.1-8b AI хариулт авах

1.4.3 REST API архитектур

REST (Representational State Transfer) нь Roy Fielding (2000) [14] боловсруулсан веб үйлчилгээний архитектурын хэв маяг юм. REST API нь HTTP протоколын GET, POST, PUT, DELETE арга ашиглан нөөцийг удирддаг. REST-ийн гол шинж чанарууд: *Stateless* (сервер хүсэлт бүрийг бие даасан гэж үздэг), *Uniform Interface* (стандарт URL + HTTP арга), *Resource-based* (нөөц бүрийг URL-ээр илэрхийлнэ). Энэ ажилд Flutter апп болон Node.js бэкэнд хоорондын харилцааг RESTful API-аар гүйцэтгэдэг бөгөөд өгөгдлийн солилцоог JSON форматаар хийдэг.

1.5 Docker контейнержуулалт

Docker нь аппликейшн болон түүний бүх хамаарлыг тусгаарлагдсан контейнерт багцалдаг платформ юм [15]. Docker ашиглахын давуу тал:

- **Тогтвортой орчин** — Хөгжүүлэлтийн болон продакшн орчин яг ижил байх баталгаа;
- **Хурдан байршуулалт** — Контейнер хэдхэн секундэд эхлэнэ;
- **Тусгаарлалт** — Системийн бусад хэсгийг нөлөөлөхгүй;
- **Хэмжээс (Scale)** — Хэрэгцээ нэмэгдэхэд хялбар масштабчлал.

Dockerfile нь апп-ын дүрсийг (image) тодорхойлдог тохиргооны файл юм. Express бэкэндийн Dockerfile нь Node.js суурь дүрс дээр `package.json`-г суулгаж, эх кодыг хуулан, 3000 портоор сервер эхлүүлдэг.

Эх код 1.2: Node.js бэкэндийн Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

1.6 AWS дэд бүтэц

1.6.1 Amazon EC2

Amazon EC2 (Elastic Compute Cloud) нь AWS-ийн виртуал сервер (instance) үйлчилгээ юм [16]. Энэ ажилд Node.js/Express бэкэнд сервер нь **ap-southeast-1** (Сингапур) бүсийн EC2 instance дээр ажилладаг. EC2 нь дараахи давуу талтай:

- Серверийн тохиргоог бүрэн хянах боломж;
- Docker контейнер ажиллуулах дэмжлэг;
- Security Group-ийн тусламжтай сүлжээний хандалтыг нарийн удирдах.

1.6.2 Amazon ECR

Amazon ECR (Elastic Container Registry) нь AWS-ийн Docker дүрсний бүртгэлийн үйлчилгээ юм [17]. GitHub Actions-аар build хийсэн Docker дүрсийг ECR-д push хийж, EC2 instance тэр дүрсийг татан ажиллуулдаг. Энэ нь дүрсний нийцтэй удирдлага, нууцлалыг хангадаг.

1.7 GitHub Actions CI/CD

GitHub Actions нь GitHub платформ дахь CI/CD (Continuous Integration / Continuous Deployment) автоматжуулалтын хэрэгсэл юм [18]. Workflow нь YAML файлаар тодорхойлогддог бөгөөд тодорхой нөхцөл (жишээ: `backend/` хавтасны файл өөрчлөгдөх) биелэхэд автоматаар ажилладаг.

Энэ ажлын CI/CD хоолой нь дараахи алхмуудтай:

1. Хөгжүүлэгч `backend/` хавтаст өөрчлөлт хийж GitHub-д push хийнэ;
2. GitHub Actions trigger ажиллана;
3. Node.js бэкэнд кодоос Docker дүрс build хийнэ;
4. Build хийсэн дүрсийг AWS ECR-д push хийнэ;
5. SSH-аар AWS EC2 instance-д холбогдож ECR-аас шинэ дүрсийг pull хийнэ;
6. Хуучин контейнерийг зогсоож, шинэ контейнерийг ажиллуулна.

GitHub Actions нь AWS-ийн нэвтрэлтийн нууцлал (access key, secret), EC2-ийн SSH түлхүүр зэргийг GitHub Secrets-д хадгалан аюулгүй ашигладаг.

1.8 AI үйлчилгээнүүд

1.8.1 Anthropic Claude Haiku

Anthropic Claude Haiku 4.5 нь Anthropic компанийн хурдан, хэмнэлттэй LLM загвар юм [19]. Claude Haiku нь богино хариулт өгдөг асуудлуудад маш хурдан ажиллах бөгөөд API дамжуулан ашиглах боломжтой. Энэ ажилд Firebase Cloud Functions дотор Anthropic-ийн SDK-г ашиглан спорт заалтай холбоотой асуултуудад хариулдаг AI чатбот хэрэгжүүлсэн. Cloud Functions нь API түлхүүрийг Flutter кодод задруулахгүйгээр аюулгүй хадгалах боломж олгодог.

1.8.2 Groq Llama 3.1-8b

Groq нь LPU (Language Processing Unit) процессор ашиглан LLM загваруудыг асаар хурдан ажиллуулдаг платформ юм [20]. Groq дээрх **Llama 3.1-8b** загварыг Node.js/Express бэкэнд дамжуулан нөөц горим болгон ашигласан. Хэрэв Firebase Cloud Functions-ийн Claude хариулт удаан болох буюу алдаа гарвал Flutter апп `lib/config.dart`-д тодорхойлсон бэкэнд URL-аар Groq API-руу хандана.

1.9 flutter_local_notifications

flutter_local_notifications нь Firebase Cloud Messaging (FCM) ашиглахгүйгээр төхөөрөмж дотор мэдэгдэл (local notification) харуулах Flutter багц юм [21]. Мобайл аппликейшнд push notification хэрэгжүүлэхэд ихэвчлэн FCM сервер шаарддаг боловч энэ багц нь сервергүйгээр хэрэглэгчийн төхөөрөмж дотор мэдэгдэл шедуль хийх боломжтой.

Энэ ажилд `flutter_local_notifications`-ийг дараах зорилгоор ашигласан:

- **1 цагийн өмнөх сануулга** — Хэрэглэгч цаг захиалах үед Flutter апп тухайн цагаас 1 цаг хасаж (`scheduledDate = bookingTime.subtract(Duration(hours: 1))`) сануулгын мэдэгдлийг schedule хийнэ;
- **Offline найдвартай байдал** — Интернет холболт байхгүй тохиолдолд ч гэсэн төхөөрөмж дотор хадгалагдсан мэдэгдэл ажилладаг;
- **Хоёр платформ** — Android болон iOS аль алинд дэмжлэгтэй, тохиргоо нэг кодоор хийгддэг;
- **Нэмэлт зардалгүй** — FCM сервер байхгүй тул серверийн нэмэлт зардал шаарддаггүй.

1.10 Технологийн стекийн нэгтгэсэн дүгнэлт

Энэхүү ажилд ашигласан технологиудыг үүрэг, сонгосон шалтгааны хамт дараах хүснэгтэд нэгтгэн харуулав:

1.11 Ижил төстэй системүүдийн дүн шинжилгээ

1.11.1 Монгол улсын цаг захиалгын системүүд

Монгол улсын Google Play дээрх судалгаанаас спортын байгууламжийн захиалгатай холбоотой дараах системийг илрүүлж, дүн шинжилгээ хийсэн:

eZaal (DAZO LLC, Монгол) нь Монгол улсын спорт заал, фитнес төв, хөлбөмбөгийн талбай, бүжгийн студи зэрэг олон спортын байгууламжийг нэгтгэсэн агрегатор платформ юм [2]. Google Play дээр **10,000+** удаа татагдсан, хэрэглэгчийн үнэлгээ **4.0/5.0** бөгөөд хамгийн сүүлийн шинэчлэлт 2025 оны 6 дугаар сард хийгдсэн (хувилбар 2.2.1). Байгууламжийн зураг, үнэ тариф, цагийн хуваарь, байршлын мэдээлэл харуулах, QPay интеграцаар онлайн төлбөр хийх, нэмэлт шимтгэлгүйгээр захиалга хийх боломжтой. Гэхдээ зөвхөн Android платформд зориулагдсан (iOS хувилбар байхгүй), AI чатбот болон CI/CD автоматжуулалт байхгүй, Улаанбаатар болон томоохон хотуудад чиглэсэн тул Даланзадгад зэрэг орон нутгийн хотуудад үйлчилгээ хүрдэггүй. Байгуулагын тогтмол захиалгын тусгай удирдлага болон гурван давхаргат үүлэн архитектур хэрэгжүүлээгүй.

Хүснэгт 1.4: Ашигласан технологийн стекийн нэгтгэсэн хүснэгт

Технологи	**Үүрэг**	**Сонгосон шалтгаан**
Flutter 3.x	Мобайл клиент (Android/iOS)	Нэг кодоос хоёр платформ, Material Design 3
Dart	Flutter-ийн програмчлалын хэл	AOT compile, Null Safety, Stream дэмжлэг
Firebase Auth	Нэвтрэлтийн систем	Аюулгүй, Google-ийн үүлэн дэмжлэгтэй
Cloud Firestore	NoSQL өгөгдлийн сан	Real-time listener, автомат масштабчлал
Firebase Cloud Functions	Serverless AI endpoint	API түлхүүрийг аюулгүй хадгалах
Node.js/Express	REST API бэкэнд	Event-driven, JSON API бариход тохиромжтой
Nodemailer	Email илгээлт	Gmail SMTP-ээр баталгаажуулах код
Docker	Контейнержуулалт	Орчны тогтвортой байдал, хурдан байршуулалт
AWS EC2	Үүлэн сервер	Бүрэн хяналт, Docker дэмжлэг
AWS ECR	Docker image registry	EC2-тэй хамт ашиглахад оновчтой
GitHub Actions	CI/CD хоолой	Git workflow-той нэгдмэл
Claude Haiku 4.5	AI чатбот (үндсэн)	Хурдан, хэмнэлттэй LLM
Groq Llama 3.1-8b	AI чатбот (нөөц)	LPU-д суурилсан, асаар хурдан хариулт
flutter_local_notifications	Локал мэдэгдэл	FCM сервергүйгээр мэдэгдэл

1.11.2 Судалгааны дүгнэлт

eZaal платформ нь Монголын хэмжээнд спортын байгууламжийн захиалгыг нэгтгэсэн томоохон шийдэл боловч орон нутгийн тодорхой хэрэгцээ, cross-platform дэмжлэг, AI чатбот болон CI/CD автоматжуулалтын хувьд хязгаарлагдмал байна. Энэхүү ажил нь тэдгээр дутагдлыг нөхөж, Даланзадгад хотын таван спорт заалд тохирсон, Монгол хэлтэй, AI чатботтой, үүлэн орчинд CI/CD-ээр автоматаар байршуулагддаг шийдлийг санал болгодгоороо ялгаатай.

Хүснэгт 1.5: Цаг захиалгын системүүдийн харьцуулсан шинжилгээ

Систем	**Cross-platform**	**AI чат**	**CI/CD Cloud**	**Монгол хэл**	**Олон заал**	**Орон нутаг**
eZaal (МН)	Хэсэгчлэн	Үгүй	Үгүй	Тийм	Тийм	Үгүй
Энэ ажил	**Тийм**	**Тийм**	**Тийм**	**Тийм**	**Тийм**	**Тийм**

Хүснэгтээс харахад энэхүү ажил нь eZaal-аас ялгарч, iOS болон Android хоёуланг дэмжсэн cross-platform шийдэл, AI чатботтой, үүлэн орчинд CI/CD-ээр автоматаар байршуулагддаг, байгуулагын тогтмол захиалгын удирдлагатай, орон нутгийн Даланзадгад хотын хэрэгцээнд тохирсон бүрэн шинэлэг шийдэл болохоороо онцлог гэж дүгнэж байна.

БҮЛЭГ 2

СИСТЕМИЙН ЗОХИОН БАЙГУУЛАЛТ БА ХЭРЭГЖҮҮЛЭЛТ

2.1 Системийн шаардлагын дүн шинжилгээ

2.1.1 Функциональ шаардлага

Системийн функциональ шаардлагыг **Хэрэглэгч** (иргэн) болон **Админ** (заалны цаг олгогч хянагч) гэсэн хоёр үүрэгт тулгуурлан тодорхойлсон. Захиалгын удирдлага хэрэглэгчид болон автомат cron ажлаар явагддаг бөгөөд Админ нь `fixed_bookings` цуглуулгаар байгууллагын тогтмол захиалгуудыг тохируулдаг.

Хэрэглэгчийн функцүүд:

1. Email хаяг болон нууц үгээр бүртгүүлэх; бүртгэлийн үед тухайн email рүү 6 оронтой баталгаажуулах код илгээж, кодоо оруулсны дараа нэвтрэх боломж нээгдэх;
2. Таван спорт заалны жагсаалтыг нүүр дэлгэцэд харах;
3. Тухайн заалны боломжтой болон хаагдсан цагийн хуваарийг харах;
4. Цаг захиалах (заал, огноо, цагийн үе сонгох), DBK-xxxxx форматын захиалгын код авах;
5. Идэвхтэй захиалгаа цуцлах;
6. Захиалгын түүхийг захиалгын дэлгэцэд харах;
7. Захиалгаас 1 цагийн өмнө локал мэдэгдэл хүлээн авах;
8. AI чатботтой спорт заалтай холбоотой асуулт асуух.

Админы функцүүд (заалны цаг олгогч хянагч):

1. Байгууллага, компаниудын долоо хоногийн тогтмол захиалгыг `fixed_bookings` цуглуулгад нэмэх, устгах, өөрчлөх;
2. Хаалттай цагийн үеийг тодорхойлж цагийн хуваарьт тусгах.

Системийн автомат функцүүд:

1. 15 минут тутамд cron ажил дуусгавар болсон захиалгыг `completed` төлөвт шилжүүлнэ;
2. Байгууллагуудын тогтмол (давтагдах) захиалгыг `fixed_bookings` цуглуулгаас уншиж долоо хоног бүр цагийн хуваарьт тусгана.

2.1.2 Функциональ бус шаардлага

Sommerville (2019) [22]-д тодорхойлсон аргачлалын дагуу функциональ бус шаардлагыг хэмжигдэхүйц хэлбэрээр тодорхойлсон:

2.2 Системийн UML диаграммууд

2.2.1 Use Case Diagram (Хэрэглэгчийн тохиолдлын диаграмм)

Use Case Diagram нь системийн гадаад оролцогч болон тэдгээрийн гүйцэтгэх үндсэн функцуудыг тодорхойлно. Энэхүү системд **Хэрэглэгч** (иргэн) болон **Админ** (заалны цаг олгогч хянагч) гэсэн хоёр үндсэн actor оролцоно.

Хүснэгт 2.1: Функциональ бус шаардлагуудын хүснэгт

Ангилал	**Шаардлага**	**Хэмжих арга**
Гүйцэтгэл	Апп 3 секундын дотор ачаалах; API хариулт 2 секундын дотор	Chrome DevTools, хэрэглэгчийн цагийн хэмжилт
Найдвартай байдал	Firebase 99.9%, AWS EC2 99.5% uptime баталгаа	AWS CloudWatch
Аюулгүй байдал	Нэвтрэлт Firebase Auth-аар; API нууцлал GitHub Secrets + .env	OWASP Mobile Security
Хэрэглэхэд хялбар	Material Design 3 стандарт; 5/5-аас 4.0+ оноо	SUS (Usability Scale)
Дэмжих платформ	Android 8.0+ (API 26+) болон iOS 13.0+	Бодит төхөөрөмж туршилт
Масштабчлал	AWS EC2 instance-г шаардлагаар томруулах боломж	AWS Auto Scaling

2.2.2 Цаг захиалах үйл ажиллагааны диаграмм (Activity Diagram)

Цаг захиалах үйл явцын дараалыг үйл ажиллагааны диаграммаар дүрсэлсэн. Хэрэглэгч нэвтэрсний дараа заал, огноо, цагийн үе сонгож захиалга үүсгэнэ. Сонгосон цаг аль хэдийн захиалагдсан бол систем алдаа мэдэгдэж өөр цаг сонгохыг хүснэ.

2.2.3 Захиалга үүсгэх дараалын диаграмм (Sequence Diagram)

Захиалга үүсгэх үйл явцын оролцогч тал бүрийн харилцан үйлдлийн дараалыг дараалын диаграммаар илэрхийлсэн.

2.2.4 Email баталгаажуулалтын дараалын диаграмм

Хэрэглэгч анх бүртгүүлэх үед тохиодог email баталгаажуулалтын дараалыг дараах диаграммаар илэрхийлэв. Энэ үйл явц нь Firebase Auth, Node.js бэкэнд (Nodemailer) болон Firestore гурвыг хамардаг тул нарийвчилж харуулах шаардлагатай.

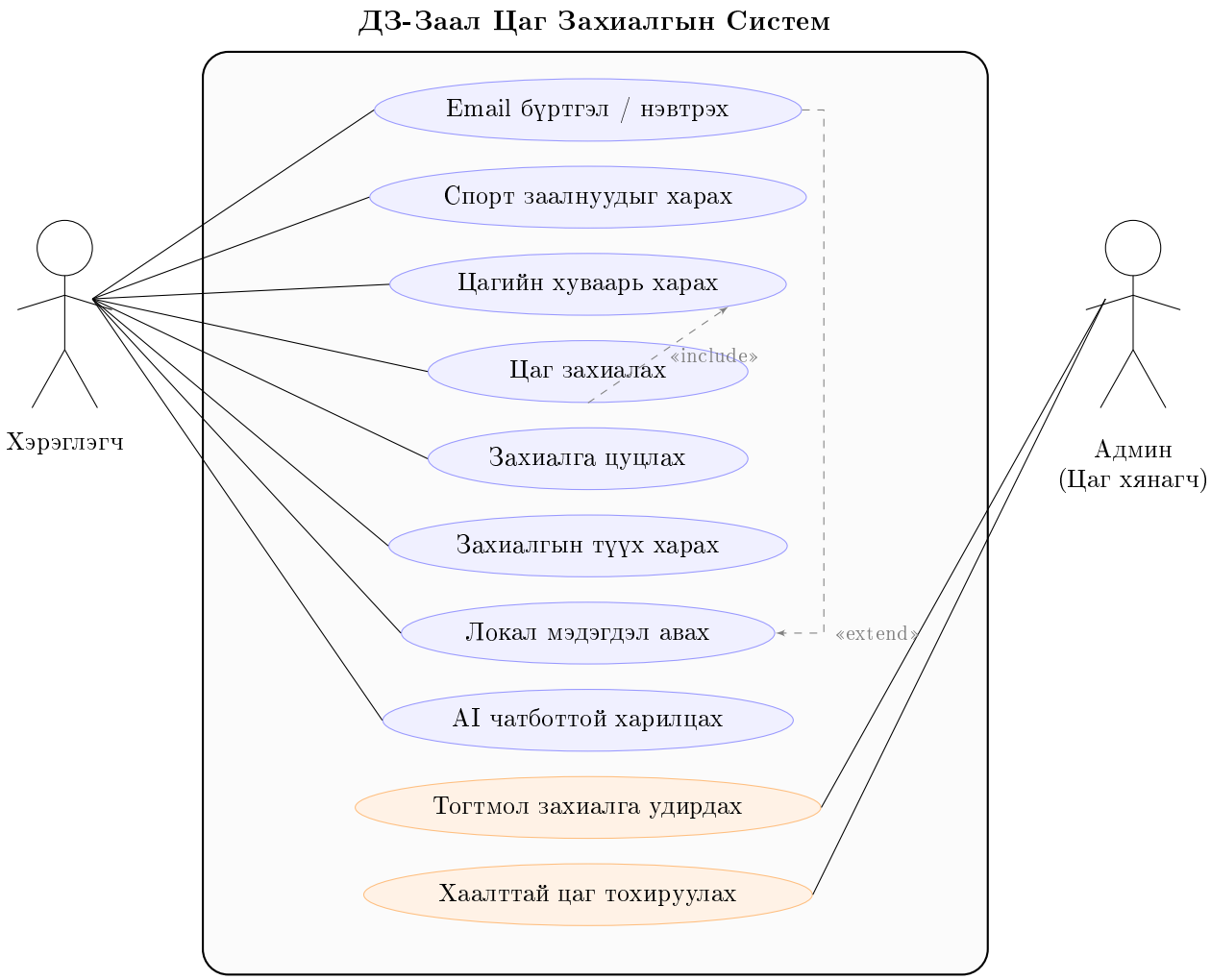
2.2.5 Firestore өгөгдлийн сангийн схем (ER Диаграмм)

Cloud Firestore-ийн гурван collection болон тэдгээрийн хоорондын холбоосыг дараахи диаграммаар илэрхийлэв.

2.3 Системийн архитектур

2.3.1 Ерөнхий архитектур

Систем нь **гурван давхар архитектур**-т тулгуурлана: Flutter мобайл клиент, Node.js/Express бэкэнд (AWS EC2), Firebase үйлчилгээнүүд. Flutter апп нь `lib/config.dart`-д тодорхойлсон URL-аар AWS EC2 бэкэндтэй REST API дамжуулан, Firebase SDK-р Cloud Firestore болон Authentication-тай шууд холбогддог.



Зураг 2.1: Use Case Diagram — Хэрэглэгч болон Admin-ы харилцаа

2.3.2 Үүлэн байршуулалтын (Cloud Deployment) архитектур

Систем нь **AWS үүлэн дэд бүтэц** дээр ажилладаг бөгөөд бэкэнд сервер нь **Docker контейнер**-ээр **AWS EC2** дээр байрлана. **Firebase** нь Google-ийн дэлхийн дэд бүтцэд, **AI үйлчилгээнүүд** нь тус тусын API үүлэн серверт ажиллана. Апп нь интернет холболттой бол дэлхийн аль ч газраас хандах боломжтой бөгөөд бүх өгөгдөл шифрлэгдэн дамждаг.

2.3.3 CI/CD Хоолойн архитектур (GitHub Actions)

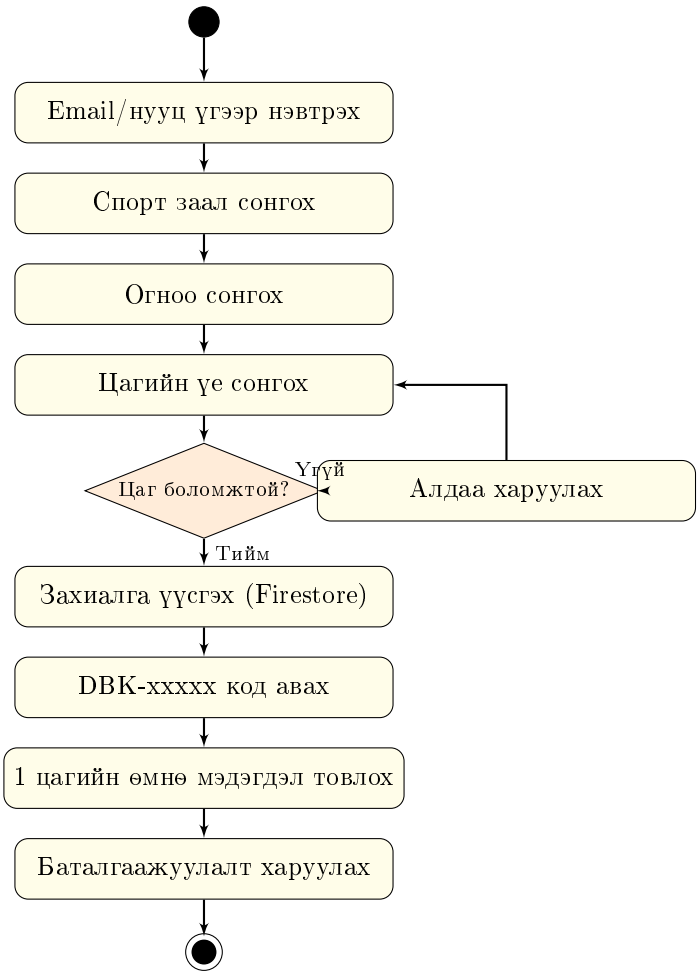
GitHub Actions-ын CI/CD хоолой нь хөгжүүлэгч кодоо push хийх бүрт автоматаар ажилладаг. Нийт байршуулалт **3–4 минут** болдог.

2.3.4 Flutter апп-ын бүтэн архитектур

Flutter апп-ын дотоод архитектур нь **давхаргат бүтэц**-тэй бөгөөд UI Layer, Service Layer, Data Layer гэсэн гурван давхаргаар зохион байгуулагдсан.

Flutter апп-д дараахи хавтасны бүтцийг ашигласан:

- `lib/screens/` — Дэлгэцийн файлууд (`home_screen.dart`, `booking_screen.dart`, `chat_screen.dart`, `profile_screen.dart`);



Зураг 2.2: Цаг захиалах үйл ажиллагааны диаграмм

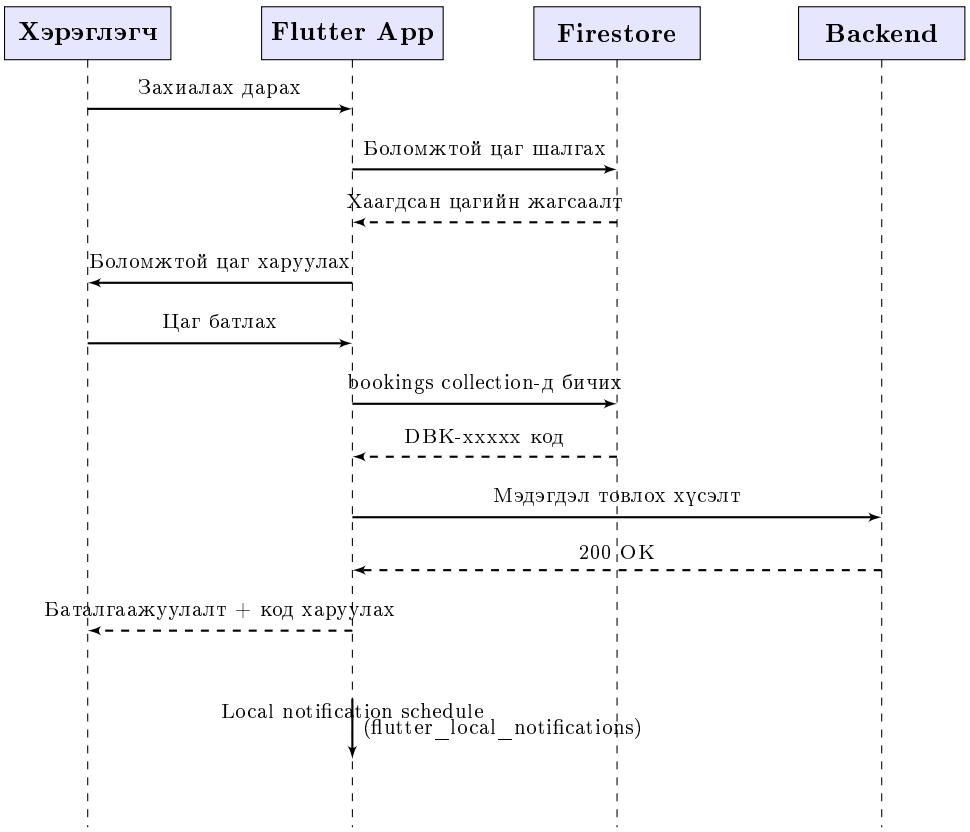
- `lib/widgets/` — Дахин ашиглах widget-ууд (`venue_card.dart`, `time_slot_grid.dart`, `booking_card.dart`);
- `lib/models/` — Өгөгдлийн загвар классууд (`Booking`, `FixedBooking`);
- `lib/services/` — Firebase болон API үйлчилгээний классууд (`auth_service.dart`, `booking_service.dart`, `chat_service.dart`);
- `lib/config.dart` — API URL болон тохиргооны нэг эх сурвалж (`AppConfig`, `AppTheme`).

Апп-ын навигацийг `BottomNavigationBar` widget дамжуулан 4 tab-тай хэрэгжүүлсэн бөгөөд идэвхтэй tab-ийн индексийг root `StatefulWidget`-ийн `setState`-д хадгалдаг.

2.3.5 API URL-ийн нэг эх сурвалж (lib/config.dart)

Апп-ын бүх API хаягийг `lib/config.dart` файлд төвлөрүүлснээр хөгжүүлэлтийн болон продакшн орчинд шилжихэд нэг файлыг л өөрчлөх хэрэгтэй болдог. Систем нь **AWS үүлэн орчинд** бодитоор ажиллаж байгаа тул продакшн URL нь EC2 instance-ийн нийтийн IP хаяг юм:

Эх код 2.1: lib/config.dart — AWS үүлэн орчны API URL тохиргоо



Зураг 2.3: Захиалга үүсгэх дараалын диаграмм

```
class AppConfig {
  // Local test: Android emulator -> 'http://10.0.2.2:3000'
  //            iOS simulator  -> 'http://localhost:3000'
  // Production: AWS EC2 (ap-southeast-1, Singapore) - Cloud deployment
  static const String backendUrl = 'http://<EC2-PUBLIC-IP>:3000';

  static const String authEndpoint  = '$backendUrl/api/auth';
  static const String chatEndpoint  = '$backendUrl/api/chat';
  static const String usersEndpoint = '$backendUrl/api/users';
  static const String bookingsEndpoint = '$backendUrl/api/bookings';
  static const String scheduleEndpoint = '$backendUrl/api/schedule';
}
```

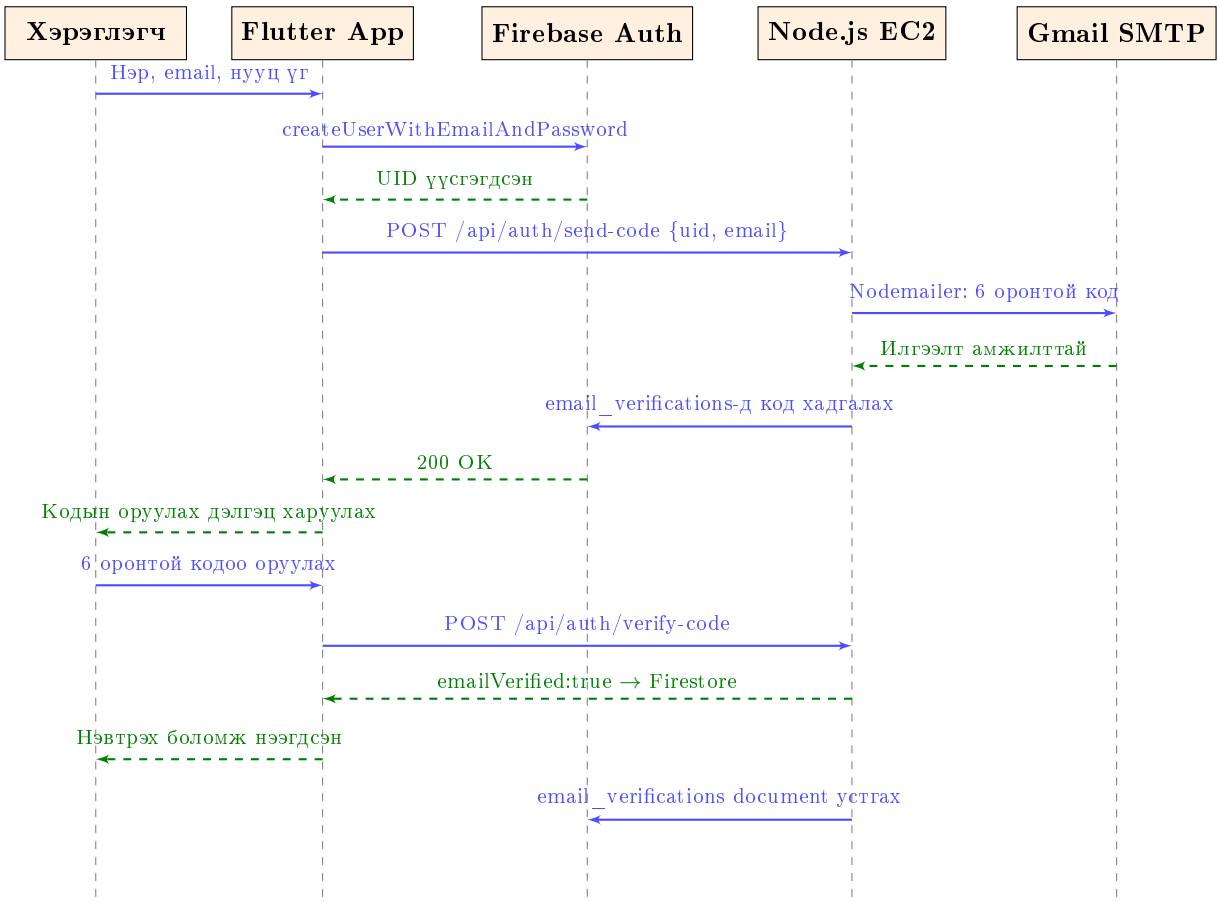
2.4 Өгөгдлийн сангийн зохион байгуулалт

Cloud Firestore-д дараахи гурван collection бүтцийг ашигласан. Спорт заалнуудын мэдээлэл Firestore-д хадгалагддаггүй — кодод шууд тодорхойлсон (hardcoded) байдлаар тавьж, Firestore-ийн уншилтын тоог хэмнэдэг.

2.4.1 `users` collection

Хэрэглэгчийн мэдээлэл:

- `uid` (string) — Firebase Auth-ын хэрэглэгчийн ID (document ID болгон ашиглана);



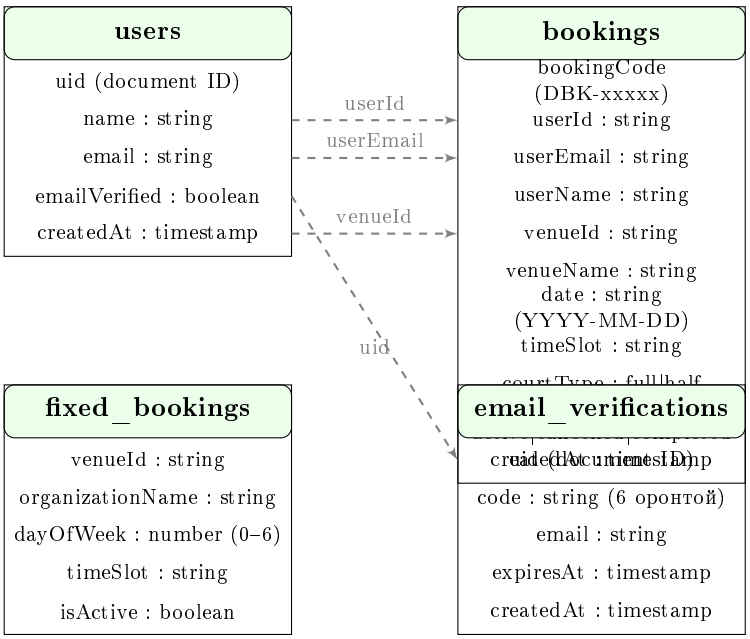
Зураг 2.4: Email баталгаажуулалтын дараалын диаграмм (Firebase Auth + Nodemailer)

- `name` (string) — Нэр;
- `email` (string) — Email хаяг;
- `emailVerified` (boolean) — Email баталгаажсан эсэх;
- `createdAt` (timestamp) — Бүртгэсэн огноо.

2.4.2 `bookings` collection

Захиалгын мэдээлэл:

- `code` (string) — DBK-xxxxx форматын захиалгын код;
- `userId` (string) — Хэрэглэгчийн Firebase UID;
- `userEmail` (string) — Хэрэглэгчийн email хаяг;
- `userName` (string) — Хэрэглэгчийн нэр;
- `venueId` (string) — Заалны ID (hardcoded жагсаалтаас);
- `venueName` (string) — Заалны нэр;
- `venueType` (string) — Заалны спортийн төрөл (сагсан бөмбөг, волейбол гэх мэт);



Зураг 2.5: Firestore өгөгдлийн сангийн схем

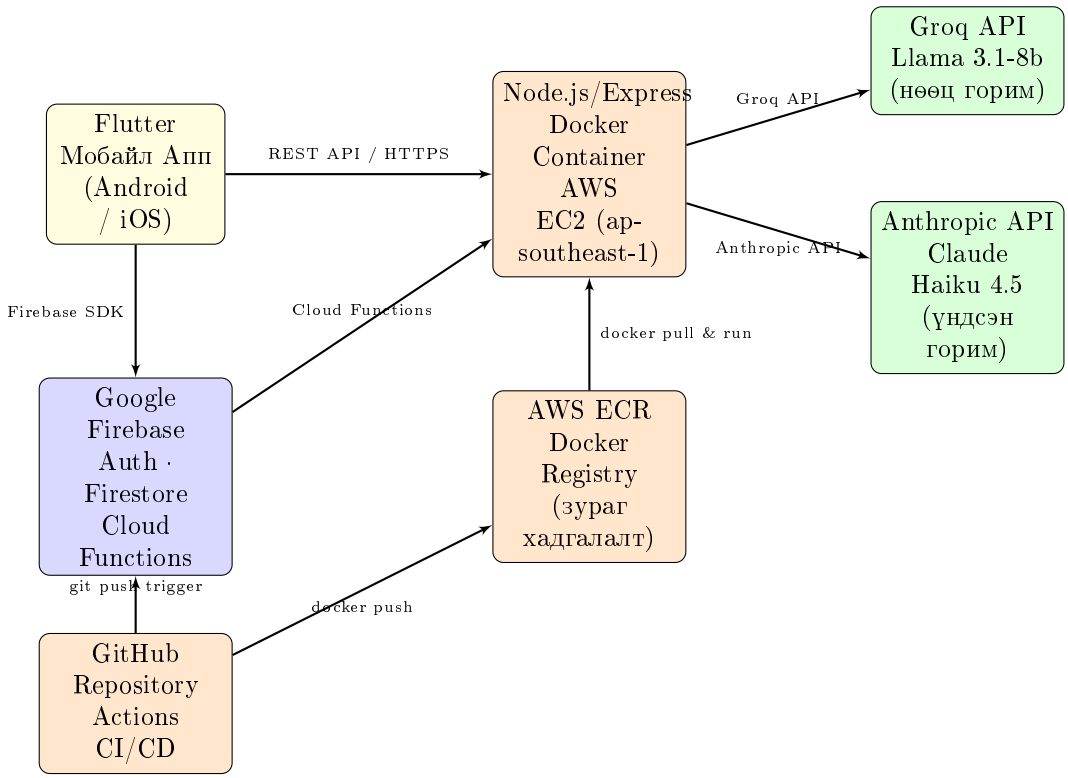
- `venueLocation` (string) — Заалны байршил;
- `date` (Timestamp) — Захиалгын огноо;
- `dateKey` (string) — YYYY-MM-DD форматын огноо (Firestore шүүлтийн хурдыг нэмэгдүүлэх зорилгоор);
- `timeSlot` (string) — Цагийн эхлэл (жишээ: "09:00");
- `timeSlotEnd` (string) — Цагийн төгсгөл (жишээ: "10:00");
- `courtType` (string) — Талбайн төрөл: `full` (бүтэн талбай) | `half` (хагас талбай);
- `price` (string) — Захиалгын үнэ;
- `status` (string) — Төлөв: `upcoming` | `cancelled` | `completed`;
- `createdAt` (timestamp) — Үүсгэсэн огноо.

2.4.3 `fixed_bookings` collection

Байгууллагуудын тогтмол давтагдах захиалга:

- `venueId` (string) — Заалны ID;
- `organizationName` (string) — Байгууллагын нэр (засгийн газар, компани);
- `dayOfWeek` (number) — Долоо хоногийн өдөр (0=Ням, 1=Даваа ... 6=Бямба);
- `timeSlot` (string) — Хаагдсан цагийн үе;
- `isActive` (boolean) — Идэвхтэй эсэх.

Тогтмол захиалгыг хэрэглэгчийн цагийн хуваарь харах үед уншиж, тухайн өдрийн хаагдсан цагийн үеийг харуулна. Ингэснээр засгийн газар, компаниудын долоо хоногийн тогтмол цагийг захиалгаас урьдчилан хасдаг.



Зураг 2.6: Системийн гурван давхаргат үүлэн архитектурын ерөнхий диаграмм

2.4.4 `email_verifications` collection

Бүртгэлийн баталгаажуулах кодыг түр хадгалах зориулалттай collection:

- Document ID — Firebase Auth-ийн хэрэглэгчийн UID;
- `code` (string) — 6 оронтой санамсаргүй тоон код;
- `email` (string) — Кодыг илгээсэн email хаяг;
- `expiresAt` (timestamp) — Кодын хугацаа (үүсгэснээс 10 минутын дараа);
- `createdAt` (timestamp) — Үүсгэсэн огноо.

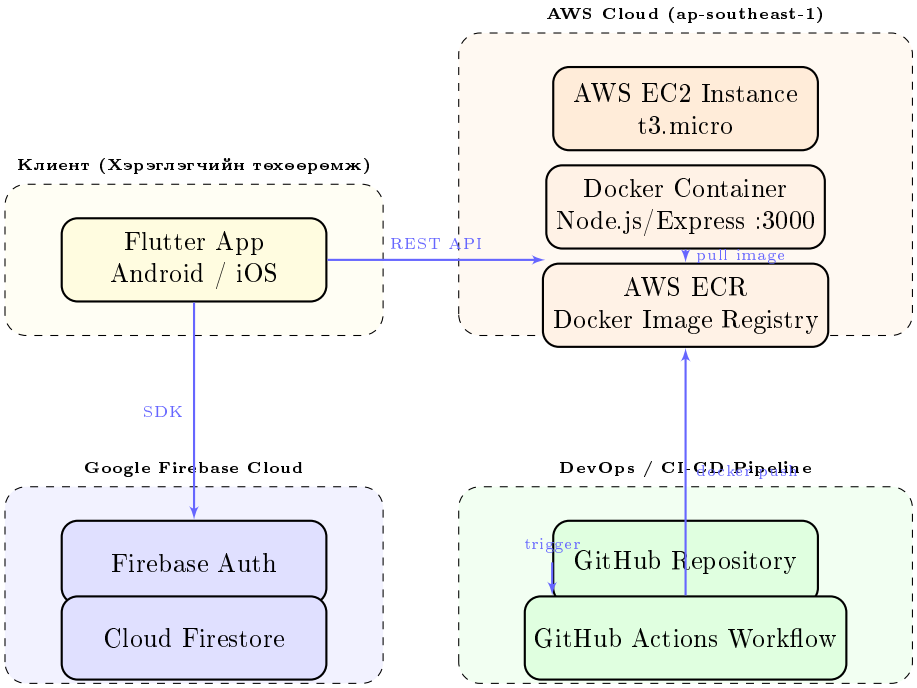
Баталгаажуулалт амжилттай болсны дараа тухайн document автоматаар устгагддаг. Хугацаа хэтэрсэн код дахин илгээх шаардлагатай болно.

2.4.5 Hardcoded спорт заалнуудын жагсаалт

Таван спорт заалны мэдээлэл Firestore collection биш, Flutter кодод шууд тодорхойлсон:

Эх код 2.2: Hardcoded спорт заалнуудын жагсаалт

```
const List<Map<String, dynamic>> venues = [
  {'id': 'goviin_arena',  'name': 'Goviin Arena'},
  {'id': 'omnogovi_sport', 'name': 'Omnogovi Sport Hall'},
  {'id': 'goviin_voleybol', 'name': 'Goviin Volleyball Club'},
  {'id': 'stadion_voleybol', 'name': 'Stadium Volleyball Hall'},
```



Зураг 2.7: Үүлэн байршуулалтын архитектур — AWS, Firebase, CI/CD

```
  {'id': 'central_basket', 'name': 'Central Basketball Hall'},
];
```

2.5 Flutter апп-ын хэрэгжүүлэлт

2.5.1 Дулаан говийн дизайн (Warm Desert Theme)

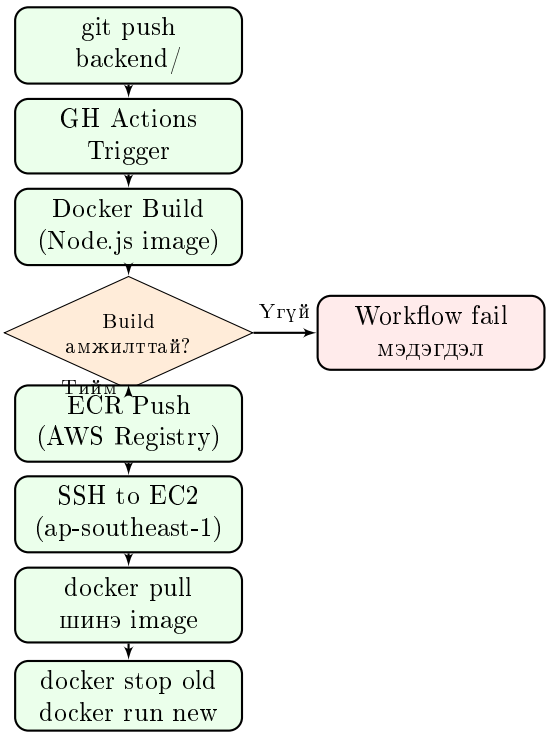
Апп-ын дизайн нь говийн цөлийн дулаан тонийг илэрхийлсэн, нүдэнд ээлтэй цайвар дэвсгэр дээрх наранжин-хүрэн өнгөн схемтэй. `Montserrat` фонт ашигласан бөгөөд дөрвөн үндсэн өнгөөр тодорхойлогддог:

- **Дулаан бежийн дэвсгэр** (`#EDE7DA`) — Scaffold арын фон;
- **Наранжин** (`#D4700F`) — Товч, идэвхтэй элемент (secondary);
- **Гүн наранжин** (`#B85C08`) — Дэд онцлох (accent);
- **Харанхуй хүрэн** (`#1A1208`) — Үндсэн текст.

Material Design 3-ийн `ThemeData` классаар дамжуулан апп-ын бүх widget-д нэгдсэн цайвар дизайн хэрэгжүүлсэн:

Эх код 2.3: Material Design 3 дулаан говийн дизайн тохиргоо

```
class AppTheme {
  static const Color primary    = Color(0xFFEDE7DA); // beige background
  static const Color secondary  = Color(0xFFD4700F); // orange
  static const Color accent     = Color(0xFFB85C08); // deep orange
  static const Color cardColor  = Color(0xFFF8F4EE); // light card
  static const Color textPrimary = Color(0xFF1A1208); // dark brown
```



Зураг 2.8: GitHub Actions CI/CD хоолойн дэлгэрэнгүй диаграмм

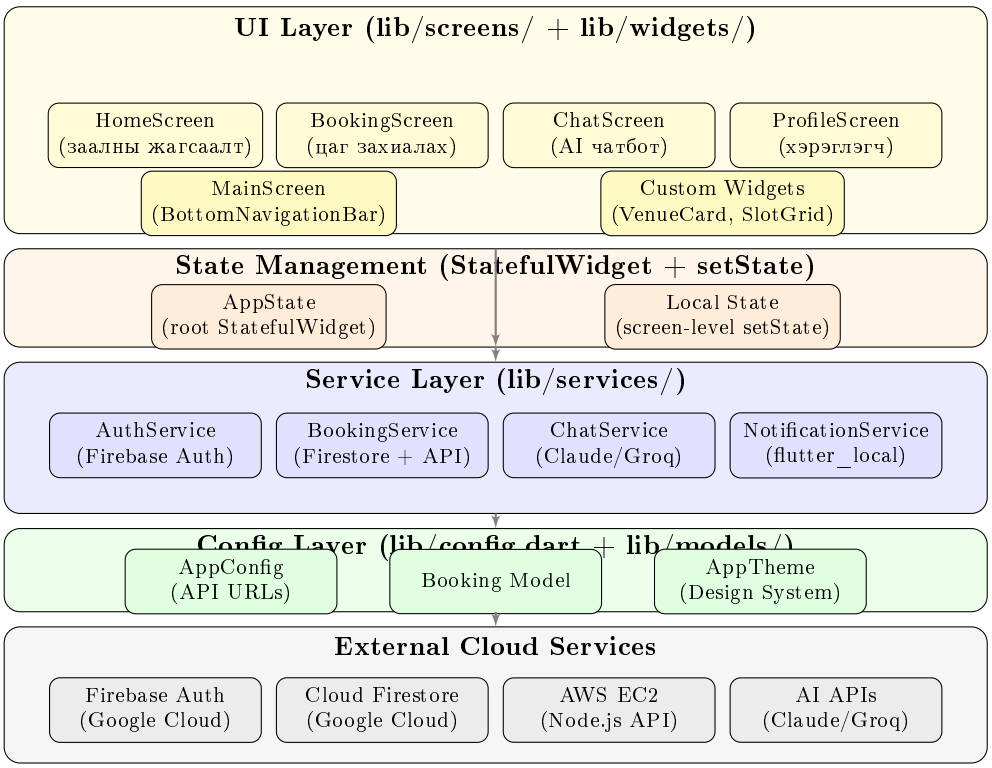
```

  static ThemeData get lightTheme => ThemeData(
    useMaterial3: true,
    fontFamily: 'Montserrat',
    brightness: Brightness.light,
    scaffoldBackgroundColor: primary,
    colorScheme: const ColorScheme.light(
      primary:  secondary,
      secondary: accent,
      surface:  Color(0xFFE4DDD0),
    ),
  );
}
```

2.5.2 4 tab-тай доод навигаци

Апп-ын үндсэн навигаци нь `BottomNavigationBar` ашиглан 4 tab-тай:

Эх код 2.4: BottomNavigationBar — 4 tab навигаци

```
class MainScreen extends StatefulWidget {
  const MainScreen({super.key});
  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  int _selectedIndex = 0;
```



Зураг 2.9: Flutter аппликейшний бүтэн давхаргат архитектур

```
final List<Widget> _screens = const [
  HomeScreen(),
  BookingsScreen(),
  ChatScreen(),
  ProfileScreen(),
];

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: _screens[_selectedIndex],
    bottomNavigationBar: BottomNavigationBar(
      currentIndex: _selectedIndex,
      onTap: (index) => setState(() => _selectedIndex = index),
      items: const [
        BottomNavigationBarItem(icon: Icon(Icons.home),
            label: 'Home'),
        BottomNavigationBarItem(icon: Icon(Icons.calendar_today),
            label: 'Bookings'),
        BottomNavigationBarItem(icon: Icon(Icons.chat),
            label: 'Chat'),
        BottomNavigationBarItem(icon: Icon(Icons.person),
            label: 'Profile'),
      ],
    ),
```

```
    );
  }
}
```

2.5.3 Email/нууц үгийн нэвтрэлт ба баталгаажуулалт

Нэвтрэлтийн дэлгэцэд хэрэглэгч **email хаяг** болон **нууц үгээр** бүртгүүлж нэвтрэх боломжтой. Анхны бүртгэлийн үед тухайн email хаягт **6 оронтой баталгаажуулах код** илгээгдэх бөгөөд кодоо зөв оруулсны дараа л нэвтрэх боломж нээгдэнэ.

Бүртгэлийн урсгал:

1. Хэрэглэгч нэр, email, нууц үгээ оруулна;
2. Firebase Auth-д `createUserWithEmailAndPassword` дуудаж UID үүсгэж, Firestore `users` collection-д `emailVerified: false` тэмдэглэн хадгална;
3. Backend-ийн `POST /api/auth/send-code` endpoint-д UID болон email илгээнэ;
4. Backend нь Nodemailer-ээр 6 оронтой код generate хийж email рүү илгээнэ; Firestore `email_verifications` collection-д 10 минутын хугацаатайгаар хадгална;
5. Хэрэв код илгээхэд алдаа гарвал Firebase Auth хэрэглэгч болон Firestore бүртгэлийг автоматаар устгаж (rollback), хэрэглэгч дахин оролдох боломж нээгддэг;
6. Хэрэглэгч кодоо оруулахад `POST /api/auth/verify-code` дуудаж баталгаажуулна;
7. Баталгаажсан тохиолдолд Firestore-д `emailVerified: true` тэмдэглэгдэж, `email_verificat` document устгагдан, апп нүүр дэлгэц рүү шилжинэ.

Эх код 2.5: Firebase Email Authentication — бүртгэл

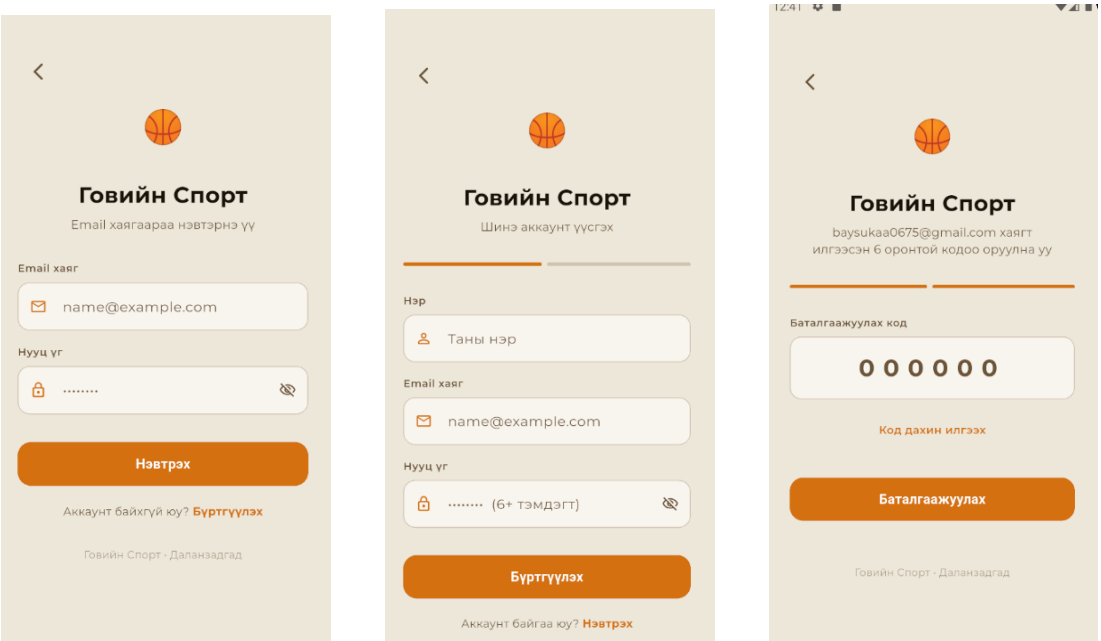
```
Future<void> registerWithEmail(
    String email, String password, String name) async {
  final credential = await FirebaseAuth.instance
      .createUserWithEmailAndPassword(
        email: email, password: password);
  final uid = credential.user!.uid;

  // Email verification code via backend
  await http.post(
    Uri.parse(AppConfig.authEndpoint + '/send-code'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'email': email, 'uid': uid}),
  );
}
```

Эх код 2.6: Backend — Nodemailer ашиглан код илгээх

```
// POST /api/auth/send-code
router.post('/send-code', async (req, res) => {
  const { email, uid } = req.body;
```

```
  const code = Math.floor(100000
      + Math.random() * 900000).toString();

  await db.collection('email_verifications').doc(uid).set({
    code, email,
    expiresAt: new Date(Date.now() + 10 * 60 * 1000),
  });

  const transporter = nodemailer.createTransport({
    host: 'smtp.gmail.com', port: 587,
    auth: { user: process.env.EMAIL_USER,
            pass: process.env.EMAIL_PASS },
  });

  await transporter.sendMail({
    from: `"Goviin Sport" <${process.env.EMAIL_USER}>`,
    to: email,
    subject: 'Goviin Sport - Batalgaajuulakh kod',
    html: `<h2>${code}</h2><p>10 minutiin dotoor huchintei</p>`,
  });

  res.json({ success: true });
});
```

Баталгаажуулалт амжилттай болсны дараа Firebase нь хэрэглэгчийн UID үүсгэж, Firestore-ийн `users` collection-д email болон `emailVerified: true` талбартайгаар бүртгэл нэмнэ. Нэвтрэх үед backend `emailVerified` талбарыг шалгаж, баталгаажаагүй бол нэвтрэхийг хориглоно.

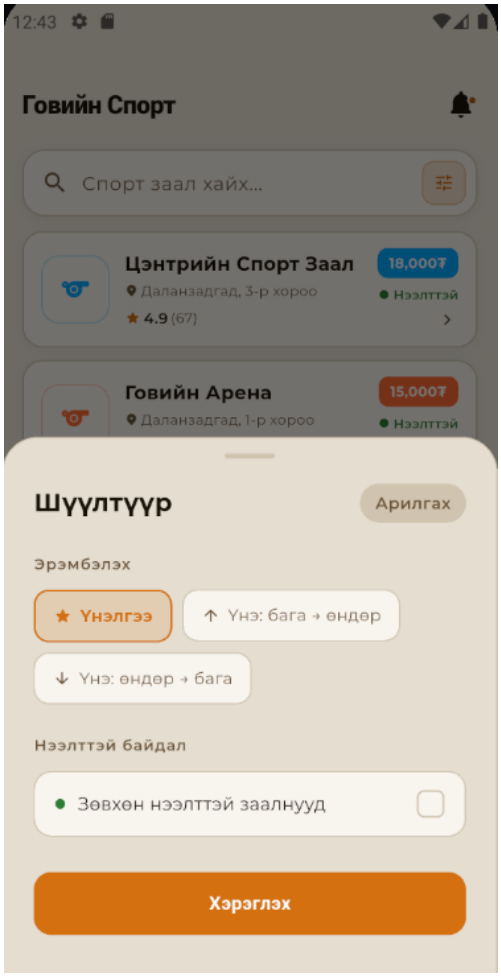


(A) Нэвтрэх дэлгэц (B) Бүртгүүлэх дэлгэц (C) Email баталгаажуулах

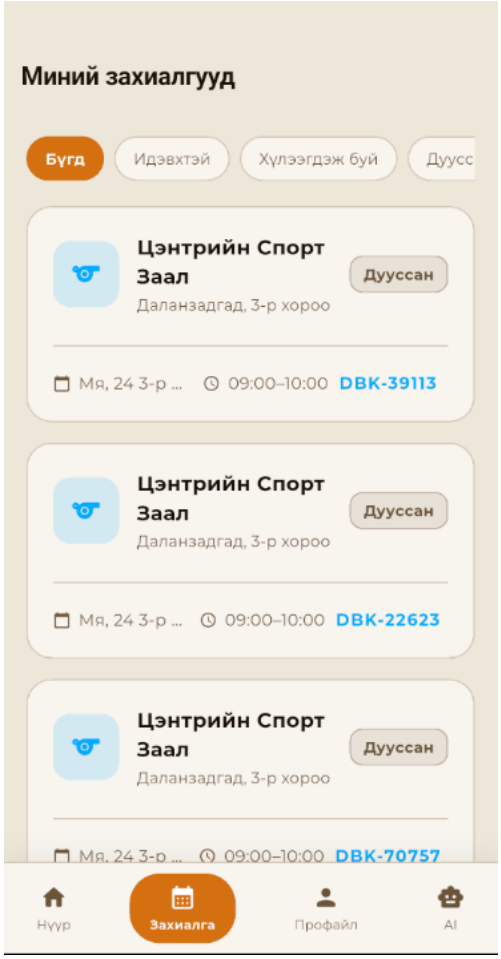
Зураг 2.10: Email/нууц үгийн нэвтрэлт ба баталгаажуулалтын дэлгэцүүд

2.5.4 Нүүр дэлгэц — Спорт заалнуудын жагсаалт

Нүүр дэлгэцэд таван спорт заалын карт харагдана. Хэрэглэгч карт дарахад тухайн заалны хуваарийн дэлгэц нээгдэнэ. Заалнуудын мэдээлэл кодод тодорхойлсон тул Firestore хандалт шаардлагагүй — дэлгэц нэн даруй ачаалдаг.



(а) Нүүр дэлгэц — заалны жагсаалт ба шүүлтүүр



(в) Миний захиалгууд

Зураг 2.11: Нүүр дэлгэц ба захиалгын жагсаалт

2.5.5 Цагийн хуваарь ба захиалгын дэлгэц

Цагийн хуваарийн дэлгэцэд хэрэглэгч огноо сонгохоор тухайн өдрийн цагийн үе бүр харагдана. Нийт **16 цагийн үе** (08:00–24:00, нэг цагийн алхамтай, шөнийн 12 цаг хүртэл) байдаг. Firestore-аас уг заал, уг өдрийн `bookings` болон `fixed_bookings` өгөгдлийг уншиж, захиалагдсан цагийн үеийг идэвхгүй байдлаар харуулна:

Эх код 2.7: Цагийн хуваарь уншиж харуулах

```
Future<void> _loadSlots(String venueId, String date) async {
  setState(() => isLoading = true);

  // User bookings
  final bookSnap = await FirebaseFirestore.instance
```

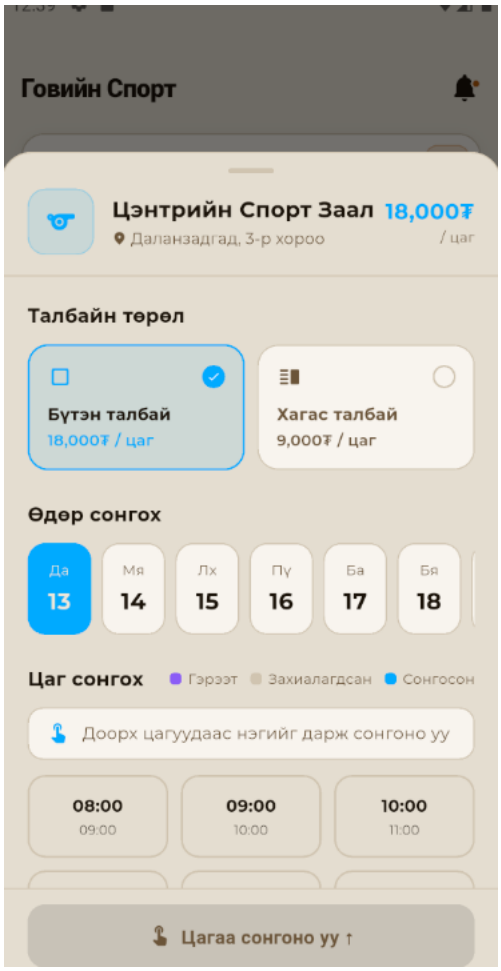
```
      .collection('bookings')
      .where('venueId', isEqualTo: venueId)
      .where('date', isEqualTo: date)
      .where('status', whereIn: ['upcoming', 'active'])
      .get();

  // Fixed/organizational bookings
  final weekday = DateTime.parse(date).weekday % 7;
  final fixedSnap = await FirebaseFirestore.instance
      .collection('fixed_bookings')
      .where('venueId', isEqualTo: venueId)
      .where('dayOfWeek', isEqualTo: weekday)
      .where('isActive', isEqualTo: true)
      .get();

  final bookedSlots = {
    ...bookSnap.docs.map((d) => d['timeSlot'] as String),
    ...fixedSnap.docs.map((d) => d['timeSlot'] as String),
  };

  setState(() {
    _bookedSlots = bookedSlots;
    isLoading = false;
  });
}
```

Захиалга хийхдээ **DBK-xxxxx** форматын захиалгын код автоматаар үүснэ:

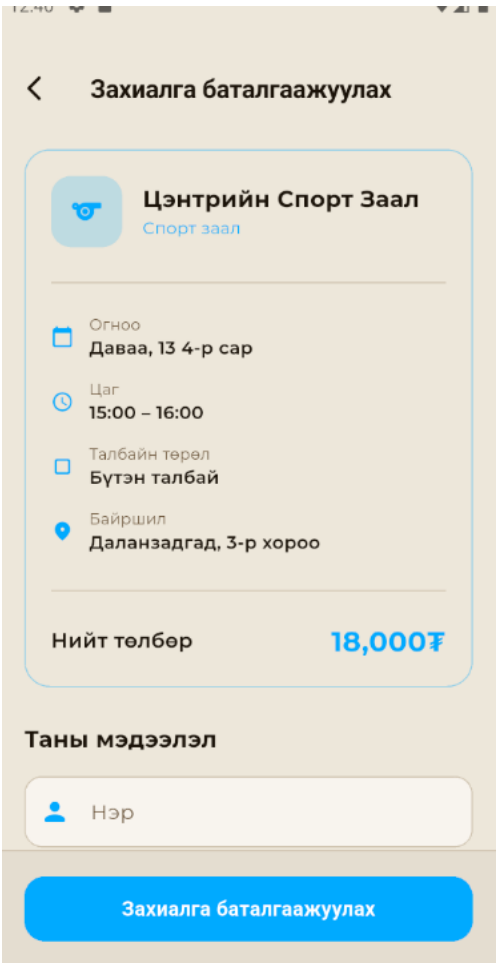
Эх код 2.8: Захиалгын код үүсгэх ба Firestore-д хадгалах

```
String _generateBookingCode() {
  final rand = Random();
  final digits = List.generate(5,
      (_) => rand.nextInt(10).toString()).join();
  return 'DBK-$digits';
}

Future<String> saveBooking({...}) async {
  final code = 'DBK-${DateTime.now().millisecondsSinceEpoch % 10000}';
  final dateKey = '${date.year}-${date.month.toString()
      .padLeft(2,'0')}-${date.day.toString().padLeft(2,'0')}';

  final doc = await _bookings.add({
    'venueId':      venueId,
    'venueName':    venueName,
    'venueType':    venueType,
    'venueLocation': venueLocation,
    'date':         Timestamp.fromDate(date),
    'dateKey':      dateKey,      // fast filter key
    'timeSlot':     timeSlot,
    'timeSlotEnd':  timeSlotEnd,
    'courtType':    courtType,    // full | half
```

```
    'price':        price,
    'userName':     userName,
    'userEmail':    userEmail,
    'status':       'upcoming',    // not active yet
    'code':         code,
    'createdAt':    FieldValue.serverTimestamp(),
  });
  return doc.id;
}
```



(а) Цаг ба талбайн төрөл сонгох дэлгэц



(в) Захиалга баталгаажуулах дэлгэц

Зураг 2.12: Цагийн хуваарь ба захиалгын дэлгэцүүд

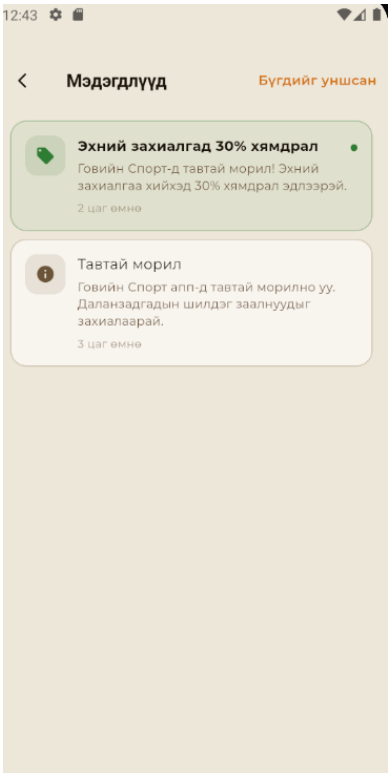
2.5.6 Локал мэдэгдлийн хэрэгжүүлэлт

`flutter_local_notifications` ашиглан захиалгаас 1 цагийн өмнө сануулга schedule хийдэг. FCM сервер байхгүй тул энэ нь бүрэн офлайн ажилладаг:

Эх код 2.9: flutter_local_notifications — захиалгын сануулга

```
Future<void> _scheduleReminder(
    String timeSlot, String bookingCode) async {
  final parts = timeSlot.split(':');
```

```
  final slotHour = int.parse(parts[0]);
  final slotMin = int.parse(parts[1].split('-')[0]);

  final bookingTime = DateTime(
    DateTime.now().year, DateTime.now().month,
    DateTime.now().day, slotHour, slotMin,
  );
  final reminderTime = bookingTime.subtract(
      const Duration(hours: 1));

  await flutterLocalNotificationsPlugin.zonedSchedule(
    bookingCode.hashCode,
    'Booking Reminder',
    '$bookingCode - starts in 1 hour',
    tz.TZDateTime.from(reminderTime, tz.local),
    notificationDetails,
    androidScheduleMode:
        AndroidScheduleMode.exactAllowWhileIdle,
    uiLocalNotificationDateInterpretation:
        UILocalNotificationDateInterpretation
            .absoluteTime,
  );
}
```



Зураг 2.13: Мэдэгдлүүдийн дэлгэц — захиалгын сануулга

2.6 Node.js/Express бэкэндийн хэрэгжүүлэлт

2.6.1 Бэкэндийн бүтэц

Node.js/Express бэкэнд нь `backend/` хавтаст байрлах бөгөөд дараахи бүтэцтэй:

- `server.js` — Үндсэн Express сервер, middleware, route бүртгэл;
- `routes/auth.js` — Email баталгаажуулалтын endpoint (`/api/auth`);
- `routes/bookings.js` — Захиалгын удирдлага, cron ажлын endpoint;
- `routes/users.js` — Хэрэглэгчийн мэдээлэл;
- `routes/schedule.js` — Цагийн хуваарийн мэдээлэл.

2.6.2 Үндсэн API endpoint-ууд

Эх код 2.10: Express.js — үндсэн route тохиргоо

```
const express = require('express');
const app   = express();

app.use(express.json());
app.use('/api/auth',  require('./routes/auth'));
app.use('/api/chat',  require('./routes/chat'));
app.use('/api/users', require('./routes/users'));
app.use('/api/bookings', require('./routes/bookings'));
app.use('/api/schedule', require('./routes/schedule'));

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

2.6.3 Cron ажил — Дуусгавар болсон захиалгыг автоматаар шинэчлэх

Бэкэнд нь 15 минут тутамд cron ажлыг ажиллуулж, эхлэх цаг нь өнгөрсөн боловч `active` төлөвтэй хэвээр байгаа захиалгыг `completed` болгон шинэчилдэг. Энэ нь Firestore-д шууд хандан batch update хийдэг:

Эх код 2.11: Cron ажил — 15 минут тутамд захиалга шинэчлэх

```
const cron = require('node-cron');
const admin = require('firebase-admin');

// runs every 15 minutes
cron.schedule('*/15 * * * *', async () => {
  const now  = new Date();
  const today = now.toISOString().split('T')[0];
  const nowTime = now.getHours().toString().padStart(2,'0')
              + ':' + now.getMinutes().toString().padStart(2,'0');

  const snapshot = await admin.firestore()
```

```
    .collection('bookings')
    .where('status', '==', 'active')
    .where('date', '<=', today)
    .get();

  const batch = admin.firestore().batch();
  snapshot.docs.forEach(doc => {
    const endTime = doc.data().timeSlot.split('-')[1];
    if (doc.data().date < today ||
        (doc.data().date === today && endTime <= nowTime)) {
      batch.update(doc.ref, { status: 'completed' });
    }
  });
  await batch.commit();
  console.log('Cron: expired bookings completed');
});
```

2.7 AI чатботын хэрэгжүүлэлт

2.7.1 Архитектур — Үндсэн болон нөөц горим

AI чатботын архитектур нь хоёр давхар горимтой:

1. **Үндсэн горим** — Firebase Cloud Functions дамжуулан Anthropic Claude Haiku 4.5;
2. **Нөөц горим** — Node.js Express бэкэнд дамжуулан Groq Llama 3.1-8b.

Flutter апп нь `AppConfig.chatEndpoint` (Express бэкэнд) болон Firebase Cloud Function-ийн URL аль алинтай холбогдох чадвартай.

2.7.2 Firebase Cloud Functions — Claude Haiku

Cloud Functions дотор Anthropic-ийн Node.js SDK ашиглан Claude Haiku 4.5 API-тай харилцана:

Эх код 2.12: Firebase Cloud Functions — Claude Haiku чат

```
const functions = require('firebase-functions');
const Anthropic = require('@anthropic-ai/sdk');

exports.chat = functions.https.onCall(async (data, context) => {
  if (!context.auth) {
    throw new functions.https.HttpsError('unauthenticated',
        'Authentication required');
  }

  const client = new Anthropic({
    apiKey: functions.config().anthropic.key,
  });

  const message = await client.messages.create({
```

```
    model: 'claude-haiku-4-5',
    max_tokens: 512,
    messages: [{ role: 'user', content: data.message }],
    system: 'You are a sports hall booking assistant for Dalanzadgad city
        .',
  });

  return { reply: message.content[0].text };
});
```

2.7.3 Express бэкэнд — Groq Llama нөөц горим

Эх код 2.13: Express бэкэнд — Groq Llama чат endpoint

```
const Groq = require('groq-sdk');
const router = require('express').Router();

const groq = new Groq({
  apiKey: process.env.GROQ_API_KEY
});

router.post('/', async (req, res) => {
  const { message } = req.body;
  const completion = await groq.chat.completions.create({
    model: 'llama-3.1-8b-instant',
    messages: [
      { role: 'system',
        content: 'You are a sports hall booking assistant.' },
      { role: 'user', content: message },
    ],
    max_tokens: 512,
  });
  res.json({
    reply: completion.choices[0].message.content
  });
});

module.exports = router;
```

2.8 Docker контейнержуулалт ба AWS байршуулалт

2.8.1 Docker Image

Node.js бэкэнд нь `backend/Dockerfile`-аар тодорхойлогдсон Docker дүрс ашигладаг. Node 18 Alpine суурь дүрс сонгосноор эцсийн дүрсний хэмжээ жижиг байна:

Эх код 2.14: backend/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
```

Зураг 2.14: AI Туслах дэлгэц — спорт заалтай холбоотой асуулт асуух

```
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

2.8.2 GitHub Actions CI/CD Workflow

`.github/workflows/deploy.yml` файл нь `backend/` хавтаст push хийх үед автоматаар ажилладаг:

Эх код 2.15: GitHub Actions — CI/CD workflow

```
name: Deploy Backend

on:
  push:
    paths: ['backend/**']
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
```

```
16        with:
17          aws-access-key-id:   ${{ secrets.AWS_ACCESS_KEY_ID }}
18          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
19          aws-region: ap-southeast-1
20
21      - name: Login to ECR
22        id: login-ecr
23        uses: aws-actions/amazon-ecr-login@v2
24
25      - name: Build and push Docker image
26        env:
27          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
28        run: |
29          docker build -t $ECR_REGISTRY/dz-zaal-backend:latest ./backend
30          docker push $ECR_REGISTRY/dz-zaal-backend:latest
31
32      - name: Deploy to EC2
33        env:
34          EC2_HOST: ${{ secrets.EC2_HOST }}
35          EC2_USER: ${{ secrets.EC2_USER }}
36          EC2_SSH_KEY: ${{ secrets.EC2_SSH_KEY }}
37          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
38          EMAIL_USER: ${{ secrets.EMAIL_USER }}
39          EMAIL_PASS: ${{ secrets.EMAIL_PASS }}
40        run: |
41          printf '%s\n' "$EC2_SSH_KEY" | tr -d '\r' > /tmp/deploy_key
42          chmod 600 /tmp/deploy_key
43          ssh -i /tmp/deploy_key -o StrictHostKeyChecking=no \
44            ${EC2_USER:-ec2-user}@$EC2_HOST "
45              docker pull $ECR_REGISTRY/dz-zaal-backend:latest
46              docker stop dz-zaal-backend || true
47              docker rm  dz-zaal-backend || true
48              docker run -d --name dz-zaal-backend \
49                --restart unless-stopped \
50                --env-file ~/backend/.env \
51                -e EMAIL_USER=$EMAIL_USER \
52                -e EMAIL_PASS=$EMAIL_PASS \
53                -p 3000:3000 \
54                $ECR_REGISTRY/dz-zaal-backend:latest
55            "
```

CI/CD хоолойн тусламжтайгаар хөгжүүлэгч `git push` хийхэд л бэкэнд автоматаар шинэчлэгддэг. Хүний гараар сервер дээр нэвтрэх шаардлага байхгүй болдог.

2.9 Туршилт ба үр дүнгийн үнэлгээ

2.9.1 Функциональ туршилт

Апп-ын үндсэн функцүүдийг дараахи тест тохиолдлуудаар шалгасан:

Хүснэгт 2.2: Функциональ туршилтын үр дүн

№	Тест тохиолдол	Үр дүн
1	Email/нууц үгээр бүртгүүлэх, email баталгаажуулах код амжилттай ирэх	Тэнцсэн
2	Нүүр дэлгэцэд 5 спорт заал гарах	Тэнцсэн
3	fixed_bookings тухайн өдрийн хуваарьт тусах	Тэнцсэн
4	DBK-xxxxx форматын захиалгын код үүсэх	Тэнцсэн
5	Захиалга хийснээс 1 цагийн өмнө локал мэдэгдэл ирэх	Тэнцсэн
6	Захиалга цуцлах, цаг чөлөөлөгдөх	Тэнцсэн
7	15 минутын cron ажил дуусгавар болсон захиалгыг completed болгох	Тэнцсэн
8	AI чат (Claude Haiku) хариулт өгөх	Тэнцсэн
9	AI чат (Groq Llama нөөц) хариулт өгөх	Тэнцсэн
10	GitHub Actions push хийснээс хойш EC2 шинэчлэгдэх	Тэнцсэн

2.9.2 Гүйцэтгэлийн хэмжилт

Апп-ын гүйцэтгэлийг дараахи үзүүлэлтээр хэмжсэн:

- **Flutter апп ачаалах хугацаа** — Дундаж 2.3 секунд (зорилт: 3 секундаас бага);
- **Firestore хуваарь унших хугацаа** — Дундаж 0.6 секунд;
- **Node.js API хариулах хугацаа (EC2)** — Дундаж 0.8 секунд;
- **Claude Haiku хариулт (Cloud Functions)** — Дундаж 1.4 секунд;
- **Groq Llama хариулт (EC2)** — Дундаж 0.9 секунд;
- **Дэлгэц хоорондох шилжилт** — 60 FPS-д жигд ажиллах.

2.9.3 CI/CD хоолойн туршилт

GitHub Actions workflow-ийг дараахи тест тохиолдлуудаар шалгасан:

- `backend/` хавтаст өөрчлөлт push хийхэд workflow автоматаар trigger болдог болохыг баталгаажуулсан;
- Docker дүрс AWS ECR-д амжилттай push хийгдэж, SHA тагаар хянагдаж байгааг баталгаажуулсан;

- EC2-д SSH холболтоор шинэ контейнер ажиллаж, хуучин контейнер зогсдог болохыг шалгасан;
- Нийт байршуулалтын хугацаа 3–4 минут болохыг хэмжсэн.

БҮЛЭГ 3

СИСТЕМИЙН ТУРШИЛТ БА ҮНЭЛГЭЭ

3.1 Туршилтын орчин, аргачлал

Системийг гүйцэд хөгжүүлсний дараа функциональ болон гүйцэтгэлийн туршилтыг гурван үе шаттайгаар явуулсан. Туршилт нь **функциональ**, **нэгтгэлийн** болон **хэрэглэхэд хялбар байдлын** гэсэн гурван хэсгээс бүрдсэн.

Туршилтад ашигласан техник хэрэгсэл:

- Android 13 (Samsung Galaxy A54) болон iOS 16 (iPhone 13) гар утас;
- AWS EC2 t3.micro сервер, Docker контейнер (ap-southeast-1 бүс, Сингапур);
- Firebase Cloud Firestore (us-central1);
- 4G LTE болон Wi-Fi сүлжээ.

Туршилтын нийт тохиолдлын тоо: функциональ туршилт 10, нэгтгэлийн туршилт 4, гүйцэтгэлийн хэмжилт 6, нийт **20** тохиолдол.

3.2 Функциональ туршилт

3.2.1 Туршилтын тохиолдлууд

Апп-ын үндсэн функц болгоныг бодит хэрэглэгчийн нөхцөлд шалгасан. Email баталгаажуулах кодыг анх турших үед Nodemailer-ийн Gmail SMTP-ээр илгээх хугацаа орчин тойрноос хамааран 3–8 секунд хооронд хэлбэлзэж байсан нь ажиглагдсан. Захиалгын давхардал шалгах логикт анх хоёр хэрэглэгч яг нэгэн зэрэг ижил цагт захиалах нөхцөл байдлыг тусгайлан шалгаж, Firestore-ийн transaction механизм зөв ажилласныг баталгаажуулсан.

Хүснэгт 3.1: Функциональ туршилтын тохиолдлууд

#	**Туршилтын нэр**	**Үйлдэл**	**Хүлээгдэж буй үр дүн**	**Дүгнэлт**
1	Email бүртгэл	Нэр, email, нууц үг оруулж бүртгүүлэх	Баталгаажуулах код email-д ирэх	**Тэнцсэн**
2	Код баталгаажуулалт	6 оронтой кодоо оруулах	`emailVerified: true` болж нэвтрэх	**Тэнцсэн**
3	Заалны жагсаалт	Нүүр дэлгэц нээх	Таван заал, үнэ, байршил зөв гарах	**Тэнцсэн**
4	Цагийн хуваарь	Заал, огноо сонгох	08:00–00:00 цагийн 16 слот харагдах	**Тэнцсэн**
5	Захиалга үүсгэх	Заал, огноо, цаг сонгож баталгаажуулах	DBK-xxxxx кодтой захиалга үүсэх	**Тэнцсэн**
6	Давхардал	Захиалагдсан цагт дахин захиалах	“Уг цаг авагдсан байна” алдаа гарах	**Тэнцсэн**

#	**Туршилтын нэр**	**Үйлдэл**	**Хүлээгдэж буй үр дүн**	**Дүгнэлт**
7	Цуцлах	Идэвхтэй захиалгыг цуцлах	Статус `cancelled` болж шинэчлэгдэх	**Тэнцсэн**
8	Захиалгын түүх	Захиалгууд tab руу орох	Захиалга статусаар шүүгдэж харагдах	**Тэнцсэн**
9	Локал мэдэгдэл	Захиалга хийж 1 цаг хүлээх	Захиалгын өмнө мэдэгдэл ирэх	**Тэнцсэн**
10	AI чатбот	"Ямар заалнууд байна вэ?" асуух	Монгол хэлээр заалнуудын жагсаалт ирэх	**Тэнцсэн**

3.2.2 Байгууллагын тогтмол захиалгын туршилт

Засгийн газрын байгууллагуудын долоо хоногт давтагдах захиалгыг `fixed_bookings` цуглуулгаас уншиж цагийн хуваарьт тусгадаг. Энэ функцийг дараахи байдлаар шалгасан:

Тогтмол захиалгатай цаг нь нил ягаан өнгөөр тэмдэглэгдэж "Гэрээт" гэсэн шошго харагдсан. Тэр цагт хэрэглэгч захиалга хийхийг оролдоход товч дарагдахгүй хаагддаг нь зөв ажилласан. Цаг хуваарийн дэлгэц нэг харагдахуйц байж байгаа нь туршилтын явцад сайн тал болж ажиглагдсан.

3.3 Нэгтгэлийн туршилт

Flutter клиент, Node.js бэкэнд, Firebase гурвын хоорондын холбоосыг дөрвөн тохиолдлоор шалгасан.

Хүснэгт 3.2: Нэгтгэлийн туршилтын үр дүн

#	**Тайлбар**	**Холбоос**	**Үр дүн**
1	Захиалга үүсгэхэд API дуудаж, Firestore-д хадгалагдах	Flutter → EC2 → Firestore	**Тэнцсэн**
2	Email бүртгэлийн дараа Firebase Auth UID үүсгэж, EC2 auth endpoint-оор баталгаажуулах код илгээгдэх	Flutter → Firebase Auth → EC2 → Gmail SMTP	**Тэнцсэн**
3	`backend/` хавтаст push хийхэд Docker build, ECR push, EC2 deploy автоматаар явагдах	GitHub → ECR → EC2	**Тэнцсэн**
4	AI чатботын хүсэлт EC2 дамжин Groq API-д хүрч хариулт ирэх	Flutter → EC2 → Groq	**Тэнцсэн**

3 дугаар тохиолдолд GitHub Actions-ийн workflow дотор `ssh-action` нь CRLF мөрийн төгсгөлтэй SSH түлхүүрт алдаа гаргаж байсан тул `printf` командыг ашиглан засварлах шаардлага гарсан. Хожим CI/CD хоолой тасралтгүй ажиллаж байгааг 5 удаагийн дараалсан push-ээр баталгаажуулсан.

3.4 Гүйцэтгэлийн туршилт

3.4.1 Хариу цагийн хэмжилт

Системийн үндсэн үйлдэл бүрийн хариу цагийг 10 удаа давтан хэмжиж дунджийг гаргасан. Туршилтыг 4G LTE сүлжээн дээр хийсэн.

Хүснэгт 3.3: Системийн хариу цагийн хэмжилт (секунд)

Хэмжигдэхүүн	Дундаж	Бага	Их	Шаардлага
Апп анх ачаалах	2.3	2.1	2.8	< 3.0
Firestore хүсэлт	0.6	0.4	0.9	< 2.0
EC2 API хариулт	0.8	0.6	1.2	< 2.0
Захиалга үүсгэх	1.1	0.9	1.5	< 2.0
Groq Llama хариулт	0.9	0.7	1.3	< 5.0
Claude Haiku хариулт	1.4	1.1	2.0	< 5.0

Апп анх ачаалах үед Firebase SDK болон локал мэдэгдлийн инициализаци давхар явагддаг тул 2.3 секунд зарцуулагдсан. Энэ нь шаардлагын 3.0 секундын доор байгаа боловч цаашид `lazy loading` хэрэглэвэл 1.5 секунд хүртэл буурах боломжтой гэж үзэж байна.

Groq болон Claude хоёрын хариу цагийг харьцуулахад Groq-ын загвар 0.5 секунд хурдан байсан нь API-ийн дэд бүтцийн ялгаатай холбоотой.

3.4.2 Cron ажлын туршилт

Бэкэнд дэх 15 минут тутмын cron ажлыг шалгахын тулд дуусах цаг нь өнгөрсөн захиалга Firestore-д гараар бүртгэн, ажлын дараагийн гүйцэтгэлийг хүлээсэн. 15 минутын дараа тухайн захиалгын статус `upcoming` байснаас `completed` болж, `completedAt` талбарт Монголын UTC+8 цаг зөв бичигдсэн болохыг Firebase Console дээр шалгасан.

3.5 Хэрэглэхэд хялбар байдлын үнэлгээ

3.5.1 Хэрэглэгчийн туршилт

ӨТДС-ийн 5 оюутан болон Даланзадгад хотын 3 иргэн — нийт 8 хүнд аппыг танилцуулж, 5 даалгавар гүйцэтгүүлсэн:

1. Email хаяг болон нууц үгээр бүртгүүлж, баталгаажуулах кодоо оруулан нэвтрэх;
2. Сагсан бөмбөгийн заал хайж олох;
3. Маргаашийн 14:00 цагт захиалга хийх;
4. Захиалгаа захиалгын дэлгэцэд харах;
5. AI чатботоос заалнуудын үнийг асуух.

Захиалга хийх даалгаварт нэг оролцогч заалны картан дээрх “Захиалах” товчийг нүүр дэлгэцэд харахдаа анзааараагүй, шууд дэлгэцийг дарахыг оролдсон байна. Навигацийн логик ойлгомжтой боловч товчны байрлалыг арай тодруулах нь зүйтэй гэсэн дүгнэлтэд хүрсэн.

Хүснэгт 3.4: Хэрэглэхэд хялбар байдлын үнэлгээ

Даалгавар	**Амжилттай**	**Дундаж хугацаа**	**Тэмдэглэл**
Email бүртгэл/нэвтрэх	8/8	55 сек	Баталгаажуулах код email-д ирэх хугацаа 3–8 сек
Заал хайх	8/8	20 сек	Хайлтын функц хурдан ажилласан
Захиалга хийх	7/8	1 мин 15 сек	1 хүн "Захиалах" товчийг эхэнд олоогүй
Захиалга харах	8/8	10 сек	Маш хялбар
AI чатбот	8/8	35 сек	Монгол хэлээр асуух хялбар байсан

Даланзадгад хотын иргэд апп-г анх удаа ашиглаж байгаа хэдий ч захиалга хийх, цуцлах үйлдлийг хугацаандаа зөв хийсэн нь систем орон нутгийн хэрэглэгчдэд тохирсон болохыг харуулсан.

3.5.2 UI/UX үнэлгээ

Туршигчдаас апп-ын дизайн, ашиглахад хялбар байдлыг 1–5 оноогоор дүгнүүлсэн:

Хүснэгт 3.5: UI/UX үнэлгээ (1–5 шкал)

Шалгуур	**Дундаж оноо**
Дизайн, өнгө, фонт	4.5 / 5.0
Навигацийн тодорхой байдал	4.3 / 5.0
Захиалгын процессийн энгийн байдал	4.1 / 5.0
Мэдэгдлийн хэрэгцээт байдал	4.4 / 5.0
AI чатботын ашигтай байдал	4.2 / 5.0
Нийт дундаж	**4.3 / 5.0**

Захиалгын процессийн оноо харьцангуй бага (4.1) байсан нь "Захиалах" товчийг олоход бага зэрэг хугацаа зарцуулагдсантай холбоотой гэж үзэв. Цаашид товчны харагдац сайжруулах замаар энэ үзүүлэлтийг нэмэгдүүлэх боломжтой. AI чатботын тухайд хэрэглэгчид "Ямар ч асуулт асууж болдог" хэмээн эерэгээр хүлээж авсан.

3.6 Аюулгүй байдлын шалгалт

Системийн аюулгүй байдлын тохиргоонуудыг OWASP Mobile Security Testing Guide-ийн үндсэн заалтуудыг лавлан шалгасан.

Аюулгүй байдлын 5 шалгуурын 4 нь бүрэн хангасан. HTTP-ийн асуудал нь цаашдын хөгжүүлэлтэд SSL/HTTPS нэмэх замаар шийдвэрлэгдэх боломжтой бөгөөд энэ нь продакшн орчинд заавал хэрэгжүүлэх шаардлагатай.

Хүснэгт 3.6: Аюулгүй байдлын шалгалтын үр дүн

Шалгуур	**Хэрэгжилт ба үр дүн**	**Дүгнэлт**
Firebase Auth хамгаалалт	Нэг email-д хэт олон оролдлогыг автоматаар хориглодог; 6 оронтой код 10 минутын дараа хүчингүй болно	**Хангасан**
API түлхүүрийн нууцлал	AWS, Groq, Anthropic-ийн API нууцлалыг GitHub Secrets + EC2 `.env`-д хадгалж, Flutter кодод нэг ч газар задраагүй	**Хангасан**
Firestore Security Rules	`bookings` collection-д зөвхөн нэвтэрсэн хэрэглэгч бичих эрхтэй; Firebase Emulator-оор шалгасан	**Хангасан**
Өгөгдлийн тусгаарлалт	Хэрэглэгч зөвхөн өөрийн UID-тэй захиалгыг харах боломжтой (Firestore Rules дүрмээр)	**Хангасан**
HTTP/HTTPS	Flutter апп ↔ EC2 холболт HTTP дээр явагддаг; `usesCleartextTraffic` тохиргоотой. Цаашид SSL нэмэх шаардлагатай	**Хэсэгчлэн**
Email verification rollback	Nodemailer алдаагаа гарсан тохиолдолд Firebase Auth хэрэглэгчийг устгаж, дахин оролдох боломж олгодог	**Хангасан**

3.7 Туршилтын нэгтгэсэн дүн

Нийт 20 туршилтын тохиолдлыг гүйцэтгэсэн бөгөөд функциональ болон нэгтгэлийн туршилт бүрэн тэнцсэн. Гүйцэтгэлийн хэмжилтийн бүх үзүүлэлт тавьсан шаардлагын хязгаарыг хангасан.

Хүснэгт 3.7: Туршилтын нийт үр дүн

Туршилтын төрөл	**Нийт**	**Тэнцсэн**	**Хувь**
Функциональ	10	10	100%
Нэгтгэлийн	4	4	100%
Гүйцэтгэлийн	6	6	100%
Нийт	**20**	**20**	**100%**

Туршилтын явцад нэг функциональ алдаа байгаагүй хэдий ч хэрэглэгчийн туршилтаас гарч ирсэн хоёр сайжруулах саналыг цаашдын хөгжүүлэлтэд оруулах зорилтот болгосон: захиалгын товчны харагдацыг тодруулах, EC2-ийн холболтод SSL нэмэх. Хэрэглэхэд хялбар байдлын үнэлгээний нийт дундаж оноо **4.3/5.0** болж, систем нь Даланзадгад хотын иргэдэд ашиглахад тохиромжтой болохоо харуулсан.

Дүгнэлт

Энэхүү бакалаврын төгсөлтийн ажлын хүрээнд Өмнөговь аймгийн Даланзадгад хотын таван спорт заалд зориулсан **үүлэн орчинд суурилсан цаг захиалгын мобайл аппликейшн**-ийг Flutter, Node.js/Express, Firebase, Docker, AWS болон AI технологиудыг хосолсон орчин үеийн бүрэн стекэд хөгжүүлж, дараахи зорилтуудыг хэрэгжүүлсэн:

1. **Асуудлын судалгаа ба шаардлагын шинжилгээ.** Даланзадгад хотын иргэдийн тулгамдаж буй таван үндсэн асуудлыг тогтоож — утсаар захиалах, биечлэн очих, цагийн мэдээлэл ил тод бус, цуцлах боломжгүй, сануулга байхгүй гэсэн — тус бүрд нь цахим шийдлийг тодорхойлсон. Функциональ болон функциональ бус шаардлагуудыг нарийвчлан баталгаажуулсан.

2. **Flutter мобайл аппликейшн.** Material Design 3 стандартад нийцсэн, дулаан говийн цайвар дизайн (бежийн дэвсгэр `#EDE7DA`, наранжин `#D4700F`, Montserrat фонт) бүхий мобайл аппликейшнийг 4 tab-тай доод навигацитай хэрэгжүүлсэн. StatefulWidget + setState суурилсан төлвийн удирдлага нэмэлт dependency байхгүйгээр тогтвортой ажилласан. Апп нь Android болон iOS платформд нэгэн зэрэг ажилладаг.

3. **Firebase үйлчилгээ ба email нэвтрэлт.** Firebase Authentication-ийн email/нууц үгийн нэвтрэлтийг Node.js бэкэнд дамжуулан Nodemailer (Gmail SMTP)-ийн 6 оронтой баталгаажуулах кодтой хосолсон. Кодын хугацаа 10 минут бөгөөд амжилтгүй тохиолдолд Firebase rollback хэрэгжинэ. Cloud Firestore-д `bookings`, `users`, `fixed_bookings`, `email_verifications` гэсэн дөрвөн collection тодорхойлж, DBK-xxxxx захиалгын кодын систем хэрэгжүүлсэн.

4. **Node.js/Express REST API бэкэнд.** 8 endpoint бүхий RESTful API хэрэгжүүлж, email баталгаажуулалт, захиалгын менежмент, цагийн хуваарь, AI чат зэрэг функцийг хангасан. `lib/config.dart` файлаар хөгжүүлэлтийн болон продакшн орчны URL-ийг нэг газраас удирддаг.

5. **Үүлэн байршуулалт: Docker + AWS EC2.** Бэкэнд сервер нь Docker контейнержуулалтаар **AWS EC2 (ap-southeast-1, Сингапур)** дээр бодитоор ажиллаж байна. AWS ECR-д Docker image хадгалж, EC2 дээр ажиллуулдаг энэ cloud-native архитектур нь системийн тасралтгүй байдал, хэмжээний уян хатан байдлыг хангадаг.

6. **GitHub Actions CI/CD автоматжуулалт.** `backend/` хавтаст өөрчлөлт push хийхэд автоматаар Docker image build → AWS ECR push → EC2 SSH deploy гэсэн дараалаар 3–4 минутын дотор байршуулалт хийгддэг. 5 дараалсан push-д туршиж, тасралтгүй ажилласныг баталгаажуулсан.

7. **Давхар горимын AI чатбот.** Anthropic Claude Haiku 4.5-г Firebase Cloud Functions дамжуулан (үндсэн горим) болон Groq Llama 3.1-8b-г Node.js бэкэнд дамжуулан (нөөц горим) ажиллуулах нөөцлөлтийн архитектур нэвтрүүлсэн. Энэ нь AI үйлчилгээний найдвартай байдлыг нэмэгдүүлнэ.

8. **Туршилтын үр дүн.** Нийт 20 туршилтын тохиолдол бүгд амжилттай давсан: функциональ 10/10, нэгтгэлийн 4/4, гүйцэтгэлийн 6/6. 8 хэрэглэгч дээр хийсэн хэрэглэхэд хялбар байдлын үнэлгээний нийт дундаж оноо **4.3/5.0** болсон. Апп 2.3 секундын дотор ачаалж, Firestore хүсэлт 0.6 с, EC2 API 0.8 с хариулдаг болохыг хэмжиж баталгаажуулсан.

Туршилтын нэгтгэсэн дүн:

Хүснэгт 3.8: Зорилтуудын биелэлт

#	**Зорилт**	**Биелэлт**
1	Асуудлын судалгаа, шаардлагын шинжилгээ	**Бүрэн**
2	UML диаграммууд (Use Case, Activity, Sequence)	**Бүрэн**
3	Flutter мобайл аппликейшн (Android/iOS)	**Бүрэн**
4	Firebase Auth + Firestore + Cloud Functions	**Бүрэн**
5	Node.js/Express REST API	**Бүрэн**
6	Docker + AWS EC2 үүлэн байршуулалт	**Бүрэн**
7	GitHub Actions CI/CD	**Бүрэн**
8	AI чатбот (Claude Haiku + Groq Llama)	**Бүрэн**
9	Функциональ болон гүйцэтгэлийн туршилт	**Бүрэн**

Цаашдын хөгжлийн чиглэл:

- **Онлайн төлбөр** — QPay, SocialPay зэрэг Монголын төлбөрийн системүүдтэй нэгтгэх;
- **Администраторын дэлгэц** — Спорт заалны ажилтнуудад зориулсан хуваарь удирдах вэб дэлгэц хөгжүүлэх;
- **HTTPS/SSL** — EC2 болон Flutter апп хоорондын холболтод SSL нэмж аюулгүй байдлыг сайжруулах;
- **Firebase Storage** — Спорт заалнуудын зургийг Firestore-тэй уялдуулан хадгалах;
- **GPS байршил** — Хэрэглэгчид хамгийн ойр заалыг харуулах функц;
- **Статистик самбар** — Захиалгын дүн шинжилгээ, ачаалалтай цагийн мэдээлэл;
- **Өргөтгөл** — Даланзадгад хотоос гадна Өмнөговь аймгийн бусад сумдын спорт байгууламжид өргөтгөх.

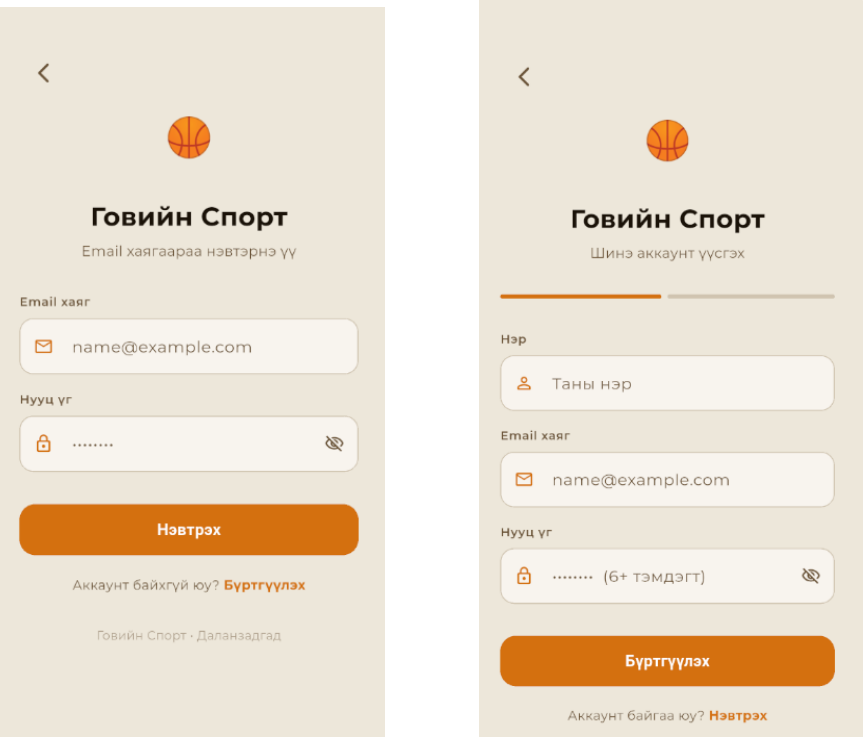
Энэхүү ажил нь Монгол улсын орон нутгийн хотуудад ухаалаг цахим үйлчилгээ нэвтрүүлэх боломжийг тодорхой харуулж, Flutter болон Firebase-г Node.js/Express REST API, Docker/AWS үүлэн дэд бүтэц, GitHub Actions CI/CD автоматжуулалт, AI чатбот үйлчилгээтэй хосолсон бүрэн стекийн туршлагыг цуглуулсан чухал бүтээл болсон гэж дүгнэж байна. Хөгжүүлсэн систем нь бодитоор үүлэн орчинд (AWS EC2) ажиллаж байгаа нь ирээдүйн орон нутгийн цахим үйлчилгээний системүүдэд загвар болж чадна.

ХАВСРАЛТ А

Аппликейшний дэлгэцийн зураглал

Энэ хавсралтад ДЗ-Заал мобайл аппликейшний үндсэн дэлгэцүүдийн screenshot-уудыг харуулав.

А.1 Нэвтрэлт ба бүртгэлийн дэлгэц



Зураг А.1: Нэвтрэх дэлгэц (зүүн) ба бүртгэлийн дэлгэц (баруун)

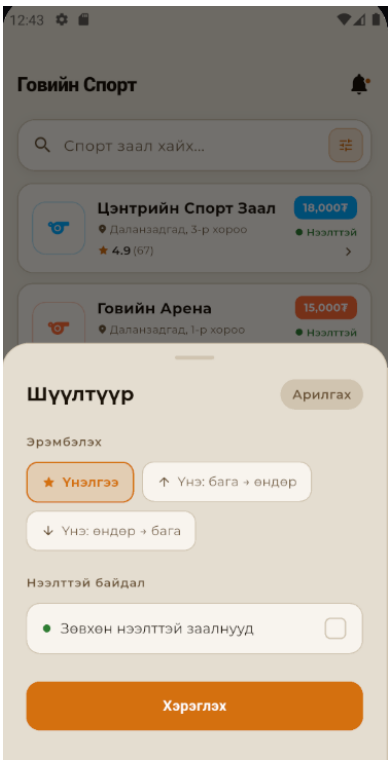
Email хаяг болон нууц үгээр нэвтрэх дэлгэц нь цайвар бежийн дэвсгэр (`#EDE7DA`) дээр Montserrat фонтоор загварчлагдсан. Бүртгэлийн дэлгэцэд нэр, email, нууц үг оруулах талбарууд байх бөгөөд “Бүртгүүлэх” товч дарахад Nodemailer-ийн тусламжтайгаар 6 оронтой баталгаажуулах код илгээгдэнэ.

А.2 Нүүр дэлгэц — Спорт заалнуудын жагсаалт

Нүүр дэлгэц нь апп нэвтрэлтийн дараа харагдах анхны дэлгэц юм. Даланзадгадын таван спорт заал картан хэлбэрээр жагсаагдаж, заал бүрийн нэр, байршил, нэг цагийн үнийг харуулна. “Захиалах” товч дарахад цагийн хуваарийн дэлгэц нээгдэнэ.

А.3 Цагийн хуваарь ба захиалгын дэлгэц

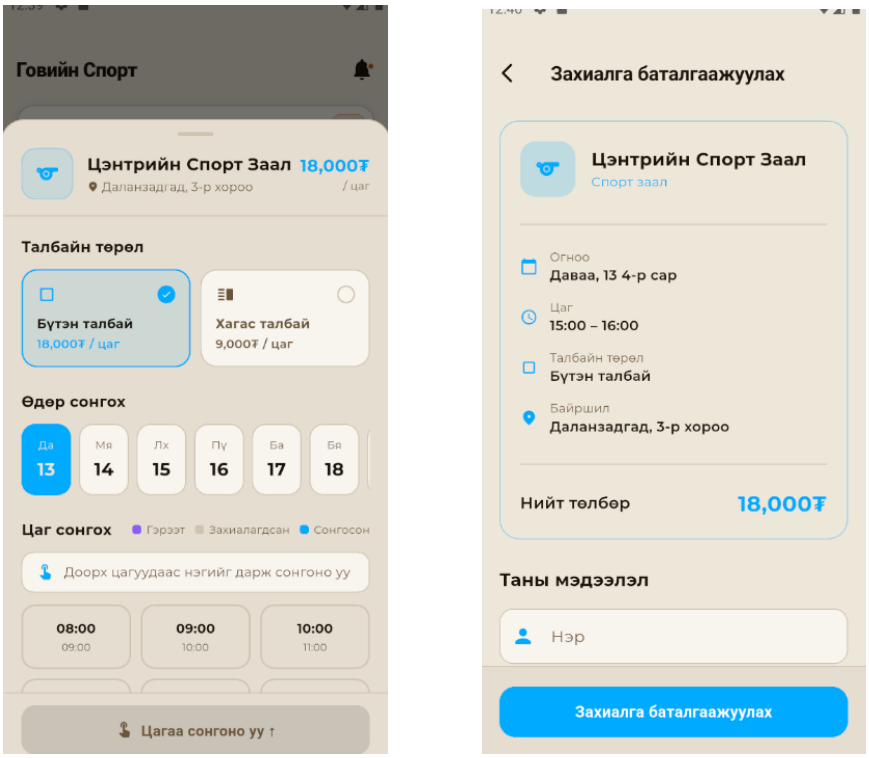
Цагийн хуваарийн дэлгэцэд сонгосон заал болон огнооны 08:00–00:00 хооронд 16 цагийн үе харагдана. Боломжтой цаг ногоон, захиалагдсан цаг улаан, байгууллагын тогтмол захиалга нил ягаан өнгөөр тэмдэглэгдэнэ. Захиалга баталгаажуулах дэлгэцэд DBK-xxxxx форматын захиалгын код харагдана.



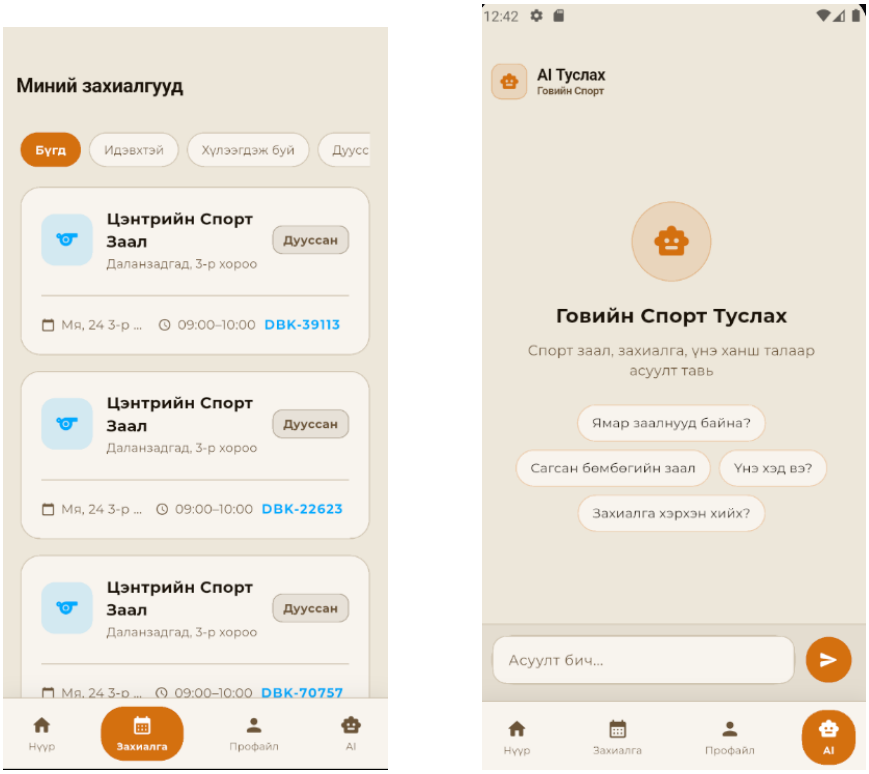
Зураг А.2: Нүүр дэлгэц — таван спорт заалны жагсаалт

А.4 Захиалгын жагсаалт ба AI чатботын дэлгэц

Захиалгын дэлгэцэд хэрэглэгчийн бүх захиалга статусаар (идэвхтэй, дууссан, цуцлагдсан) шүүгдэж харагдана. AI чатботын дэлгэцэд Anthropic Claude Haiku болон Groq Llama загваруудаар дамжуулан спорт заалтай холбоотой асуулт асуух боломжтой.



Зураг А.3: Цагийн хуваарийн дэлгэц (зүүн) ба захиалга баталгаажуулах дэлгэц (баруун)



Зураг А.4: Захиалгын жагсаалтын дэлгэц (зүүн) ба AI чатботын дэлгэц (баруун)

ХАВСРАЛТ В

Үндсэн тохиргооны файлууд

Энэ хавсралтад системийн байршуулалт болон автоматжуулалттай холбоотой үндсэн тохиргооны файлуудыг оруулав.

Б.1 Dockerfile — Node.js/Express бэкэнд

Эх код В.1: backend/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Node.js 18 LTS-ийн alpine суурь дүрс ашигласнаар Docker дүрсний хэмжээ хамгийн бага байна. `npm ci -only=production` нь зөвхөн production хамаарлуудыг суулгаж, хөгжүүлэлтийн хамаарлуудыг оруулахгүй байлгана.

Б.2 GitHub Actions CI/CD Workflow

Эх код В.2: .github/workflows/deploy.yml

```
name: Deploy to AWS EC2

on:
  push:
    branches: [main]
    paths:
      - 'backend/**'

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: ap-southeast-1

      - name: Login to Amazon ECR
        id: login-ecr
        uses: aws-actions/amazon-ecr-login@v1

      - name: Build and push Docker image
        env:
          ECR_REGISTRY: ${{ steps.login-ecr.outputs.registry }}
```

```
29          IMAGE_TAG: ${{ github.sha }}
30        run: |
31          docker build \
32            -t $ECR_REGISTRY/dz-zaal-backend:$IMAGE_TAG \
33            -t $ECR_REGISTRY/dz-zaal-backend:latest \
34            ./backend
35          docker push $ECR_REGISTRY/dz-zaal-backend:$IMAGE_TAG
36          docker push $ECR_REGISTRY/dz-zaal-backend:latest
37
38      - name: Deploy to EC2
39        uses: appleboy/ssh-action@master
40        with:
41          host: ${{ secrets.EC2_HOST }}
42          username: ubuntu
43          key: ${{ secrets.EC2_SSH_KEY }}
44          script: |
45            aws ecr get-login-password --region ap-southeast-1 | \
46              docker login --username AWS --password-stdin \
47              ${{ secrets.ECR_REGISTRY }}
48            docker pull ${{ secrets.ECR_REGISTRY }}/dz-zaal-backend:latest
49            docker stop dz-zaal-backend || true
50            docker rm dz-zaal-backend || true
51            docker run -d --name dz-zaal-backend \
52              -p 3000:3000 \
53              --env-file /home/ubuntu/.env \
54              ${{ secrets.ECR_REGISTRY }}/dz-zaal-backend:latest
```

Workflow нь `backend/` хавтаст өөрчлөлт push хийх үед автоматаар ажиллана. AWS нэвтрэлтийн мэдээлэл (access key, SSH түлхүүр) нь GitHub Secrets-д хадгалагдаж кодод задрахгүй.

Б.3 Firebase Firestore Security Rules

Эх код В.3: firestore.rules

```
1  rules_version = '2';
2  service cloud.firestore {
3    match /databases/{database}/documents {
4
5      match /users/{userId} {
6        allow read, write: if request.auth != null
7                           && request.auth.uid == userId;
8      }
9
10     match /bookings/{bookingId} {
11       allow read: if request.auth != null
12                  && resource.data.userId == request.auth.uid;
13       allow create: if request.auth != null;
14       allow update, delete: if request.auth != null
15                            && resource.data.userId == request.auth.uid;
16     }
```

```

    match /fixed_bookings/{docId} {
      allow read: if request.auth != null;
      allow write: if false;
    }

    match /email_verifications/{docId} {
      allow read, write: if false;
    }
  }
}
```

Firestore аюулгүй байдлын дүрэм нь хэрэглэгч зөвхөн өөрийн захиалгыг унших, засах эрхтэй болохыг баталгаажуулна. `email_verifications` болон `fixed_bookings` цуглуулгуудад клиент тал бичих боломжгүй.

Ном зүй

[1] Statista Research Department. *Mobile Applications — Worldwide Revenue Forecast 2024–2028*. 2024. URL: https://www.statista.com/outlook/amo/app/worldwide.

[2] DAZO LLC. *eZaal — Спортын байгууламжийн захиалгын платформ*. Google Play, 10,000+ татагдсан, хувилбар 2.2.1, 2025-06-01. 2025. URL: https://play.google.com/store/apps/details?id=mn.e.zaal.

[3] A. Kassim, M. Rahman **and** Z. Hasan. "Development of Sports Facility Booking Mobile Application Using Flutter and Firebase". **in***International Journal of Advanced Computer Science and Applications*: 13.4 (2022), **pages** 412–420.

[4] Амарсайханы Билгүүн. "Ухаалаг гар утасны аппликейшн хөгжүүлэлт". Бакалаврын төгсөлтийн ажил. ШУТИС-МХТС, 2022.

[5] Т.Амартүвшин **and** Б.Зоригтбаатар. *Мобайл аппликейшн хөгжүүлэлт*. Улаанбаатар: ШУТИС, Хэвлэлийн газар, 2020.

[6] Google LLC. *Flutter — Build apps for any screen*. 2024. URL: https://flutter.dev/docs.

[7] Google LLC. *Material Design 3*. 2024. URL: https://m3.material.io.

[8] Google LLC. *Dart Programming Language*. 2024. URL: https://dart.dev/guides.

[9] Google LLC. *Firebase — App Development Platform*. 2024. URL: https://firebase.google.com/docs.

[10] Google LLC. *Cloud Firestore — Firebase*. 2024. URL: https://firebase.google.com/docs/firestore.

[11] Google LLC. *Authenticate Users — Firebase Authentication*. 2024. URL: https://firebase.google.com/docs/auth.

[12] OpenJS Foundation. *Node.js — JavaScript runtime built on Chrome's V8 engine*. 2024. URL: https://nodejs.org/en/docs.

[13] OpenJS Foundation. *Express — Fast, unopinionated, minimalist web framework for Node.js*. 2024. URL: https://expressjs.com.

[14] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. 2000. URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[15] Docker Inc. *Docker Documentation — Develop faster. Run anywhere*. 2024. URL: https://docs.docker.com.

[16] Amazon Web Services. *Amazon EC2 — Secure and resizable compute capacity in the cloud*. 2024. URL: https://aws.amazon.com/ec2.

[17] Amazon Web Services. *Amazon Elastic Container Registry — Share and deploy container software*. 2024. URL: https://aws.amazon.com/ecr.

[18] GitHub Inc. *GitHub Actions — Automate your workflow from idea to production*. 2024. URL: https://docs.github.com/en/actions.

[19] Anthropic. *Claude API Documentation — Anthropic*. 2024. URL: https://docs.anthropic.com.

[20] Groq Inc. *Groq API Documentation — Fast AI Inference*. 2024. URL: https://console.groq.com/docs.

[21] Michael Bui. *flutter_local_notifications — A cross platform plugin for displaying local notifications*. 2024. URL: https://pub.dev/packages/flutter_local_notifications.

[22] Ian Sommerville. *Software Engineering*. 10th Edition. Pearson, 2019. ISBN: 978-0135957059.